

Data Matrix Decoding in machined components



Data Matrix Decoding in machined components

Master Thesis

February, 2026

By

Antonios Pavlou

Copyright: Reproduction of this publication in whole or in part must include the customary bibliographic citation, including author attribution, report title, etc.

Cover photo: AI generated picture, 2026

Published by: DTU, Department of Applied Mathematics and Computer Science,
Building 324, 2800 Kgs. Lyngby Denmark
www.compute.dtu.dk

Abstract

Data Matrix Codes are compact, high-density two-dimensional codes, capable of storing significant data. In industrial manufacturing, such codes are often applied directly to metal components using permanent marking techniques such as laser engraving or dot-peen marking, where symbols are formed by mechanically indenting the surface with a hardened pin. Decoding directly marked Data Matrix Codes is a challenging task due to surface reflections, mechanical degradation, low contrast, and limited availability of annotated training data. These challenges are particularly pronounced for laser-marked and dot-peened symbols captured in uncontrolled industrial imaging conditions.

This thesis proposes a geometry-first decoding pipeline that combines learning-based detection with explicit geometric and structural processing. A segmentation-based detector is used to localize the symbol region, followed by mask-driven geometric rectification, border fitting, grid reconstruction, and ECC200-compliant decoding. The pipeline enforces structural constraints at each stage and applies conservative validation to prevent error propagation and unreliable outputs.

The proposed approach is evaluated on real industrial images containing laser-marked and dot-peened Data Matrix Codes collected in collaboration with Everllence, where reliable decoding is required for component traceability in production environments. Stage-wise experimental results demonstrate robust detection performance and highlight grid fitting and decoding as the primary sources of failure under severe physical degradation. Despite these challenges, the pipeline achieves reliable decoding on a validated subset of images with known ground truth, while maintaining predictable and interpretable failure behavior.

The results show that combining learning-based localization with geometry-driven reconstruction provides a practical and transparent solution for industrial Data Matrix decoding, particularly in scenarios where data is limited and symbol structure is known.

Acknowledgements

Rasmus Larsen, Professor, Department of Mathematics and Computer Science

I would like to thank Rasmus Larsen for his supervision, guidance, and constructive feedback throughout this project. His insight and support were invaluable in shaping both the technical direction and the overall structure of the thesis.

Hans Martin Kjer, Associate Professor, Department of Mathematics and Computer Science

I would like to thank Hans Martin Kjer for his assistance and helpful discussions during the course of the project, as well as for his feedback and support when refining technical aspects of the work.

Aidan Stocks, Industry Advisor, Everllence

I would like to thank Aidan Stocks for his technical guidance and support throughout the project. His prior experience with a similar thesis, his insights from the industrial perspective and technical discussions were valuable.

Contents

Abstract	iii
Acknowledgements	iv
1 Introduction	1
1.1 Background	1
1.2 Problem Context	4
1.3 Problem Statement	5
1.4 Objectives and Scope	5
1.5 Research Questions	6
1.6 Contributions	7
1.7 Thesis Structure	7
2 Background Theory and Related Work	8
2.1 Image-Based DMC Decoding	8
2.2 Related Prior Work	8
2.3 Background Theory	11
3 Proposed Decoding Pipeline	13
3.1 Overview of the Proposed Pipeline	13
3.2 Data Acquisition and Datasets	14
3.3 Synthetic Data Generation	17
3.4 Preprocessing and Contrast Normalization	19
3.5 Learning-Based DMC Detection and Segmentation	21
3.6 Rectification from Segmentation Masks	23
3.7 Geometric Reasoning and Grid Fitting	27
3.8 Decoding	34
3.9 Implementation Details	35
3.10 Summary	36
4 Evaluation Setup and Metrics	37
4.1 Evaluation protocol	37
4.2 Detection Performance	38

4.3	Geometric rectification performance	39
4.4	Grid fitting and synthetic reconstruction performance	40
4.5	Performance on the validated decoding dataset	41
5	Discussion	44
5.1	Analysis of Failure Cases	44
5.2	Design Choices and Trade-offs	48
5.3	Limitations	53
5.4	Future Work	53
6	Conclusion	55
	Bibliography	56
A	Sudo-code of the pipeline scripts	59
A.1	Code Availability	61
B	Detection Model	62
B.1	YOLO Detection and Segmentation	62
B.2	Effect of synthetic training data	64
B.3	Effect of inverted training data	65
B.4	Detection model size	66
B.5	Overall results	67
B.6	Final model specifications and results	68
C	Geometric Rectification	70
C.1	Morphological mask cleaning	70
C.2	Douglas-Peucker simplification	71
C.3	Convex Hull	72
D	Decoding library	73
E	Process Visualization and Errors	74

1 Introduction

In industrial engineering environments, Data Matrix Codes are commonly marked directly onto machined components to enable reliable serialization and traceability throughout a product's lifecycle. These codes are typically applied using laser marking or dot-peen marking techniques, producing symbols that must remain readable despite exposure to wear, surface damage, and challenging imaging conditions. While many commercially available Data Matrix scanners perform well under controlled conditions, their reliability often degrades when used in industrial settings, particularly when images are captured using low-cost mobile devices. This gap between ideal scanning conditions and practical low-cost deployment motivates the need for more robust decoding approaches tailored to industrial use cases.

1.1 Background

1.1.1 Data Matrix Codes in Industrial Traceability

Data Matrix Codes (DMCs) are two-dimensional barcodes widely used in industrial settings for component identification and traceability. Compared to one-dimensional barcodes, DMCs offer higher data density and increased robustness to physical damage, making them suitable for marking small mechanical parts. The ECC200 standard further enhances robustness by incorporating error correction, allowing partial recovery of encoded information even when parts of the code are degraded.

In industrial manufacturing, DMCs are commonly used to serialize components so that their origin, production batch, and lifecycle information can be verified throughout supply chains. This work is conducted in collaboration with Everllence, where reliable decoding of directly marked Data Matrix Codes is required for component traceability in industrial production environments. The imaging conditions, marking quality, and symbol placement reflect practical constraints encountered on the shop floor rather than controlled inspection setups.



Figure 1.1: Engraved DMCs on industrial components.

1.1.2 Data Matrix Codes and Decoding Principles

Data Matrix Codes are two-dimensional matrix barcodes designed for compact and robust data storage. In industrial applications, DMCs are most commonly encoded according to the ECC200 standard, which defines both the symbol structure and the associated error correction scheme. ECC200 employs Reed–Solomon error correction, enabling reliable decoding even when parts of the symbol are degraded or missing [1].

Symbol size	Data codewords	ECC codewords	Max. correction
14×14	16	10	28–39%
16×16	24	16	25–38%

Table 1.1: Data capacity and error correction capability for the Data Matrix symbol sizes considered in this work, according to the ECC200 specification (adapted from [2]).

A Data Matrix symbol consists of a square grid of modules arranged into distinct functional regions, as illustrated in Figure 1.2. Two adjacent solid borders form the finder pattern, which provides orientation and alignment information, while the remaining borders follow an alternating pattern that defines the timing structure. The interior of the symbol contains data and error correction codewords arranged according to a fixed mapping specified by the standard. Accurate decoding therefore relies on correct recovery of the underlying grid and module positions.

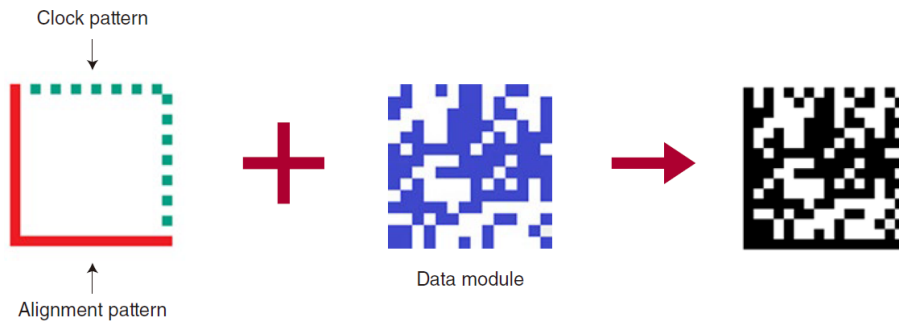


Figure 1.2: Structural components of a Data Matrix Code, including the finder pattern, timing pattern, and data region (from [2]).

In addition to the symbol itself, the ECC200 specification requires a quiet zone surrounding the code, defined as a margin free of markings with a minimum width of one module on all sides (Figure 1.3). The quiet zone supports reliable localization by separating the symbol from its background. In industrial settings, this requirement is often violated due to space constraints, surface features, or nearby engravings, increasing the difficulty of robust detection.

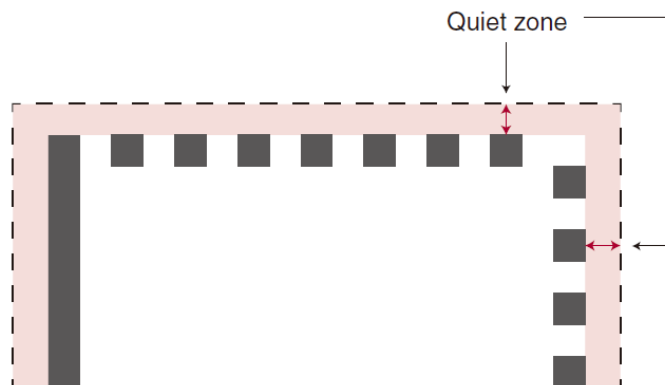


Figure 1.3: Illustration of the quiet zone surrounding a Data Matrix Code, which must remain free of markings to ensure reliable detection (from [2]).

1.1.3 Industrial Marking Techniques

To permanently attach Data Matrix Codes to metal components, industrial marking techniques such as laser marking and dot-peen marking are commonly employed. Laser marking modifies the surface of the material through localized heating, producing relatively continuous and visually well-defined module boundaries. Dot-peen marking, in contrast, relies on a mechanical process in which a hardened pin repeatedly impacts the surface to form a pattern of individual indentations representing the code modules. The quality of dot-peen markings depends not only on the material

properties of the component, but also on the mechanical precision, calibration, and wear state of the marking equipment used during engraving.

In industrial environments, dot-peen marking is often preferred due to its lower cost and robustness across a wide range of materials. However, the resulting markings are inherently variable. Individual dots may differ in size and depth, their spacing may deviate from the ideal grid due to mechanical tolerances, and surface characteristics such as scratches, machining marks, and local texture can interfere with the visual appearance of the code. Together, these factors can lead to inconsistent contrast, missing or merged dots, and distortions of the underlying grid structure, all of which complicate subsequent image-based decoding.

1.2 Problem Context

1.2.1 Traditional DMC Decoding Assumptions

Traditional Data Matrix Code decoding is based on the structural assumptions defined by the ECC200 specification. From an image-based perspective, decoding requires reliable localization of the symbol, geometric rectification to compensate for perspective distortion, recovery of its orientation and module grid, and correct sampling of individual modules according to their logical positions. These steps implicitly assume a sufficiently regular grid structure, detectable finder and timing patterns, and a clear separation between the symbol and its background.

Under controlled imaging conditions, such assumptions are typically satisfied, and standard decoding approaches can reliably extract the encoded information. However, these assumptions describe an idealized symbol appearance that is not always representative of industrially marked components.

1.2.2 Limitations in Industrial Imaging Environments

In industrial settings, the visual conditions required by traditional decoding approaches are frequently violated. Variations introduced by the marking process, surface texture, reflectance, and spatial constraints can distort the appearance of the symbol or interfere with the required quiet zone. For dot-peen marked codes in particular, modules are represented by discrete indentations rather than continuous regions, leading to

irregular spacing and ambiguous module boundaries.

As a result, failures in grid recovery or module sampling may occur even when the encoded symbol remains structurally valid. In practice, robust decoding in such environments often relies on dedicated laser-based readers, which are effective but costly and typically restricted to fixed inspection setups. The limited availability of flexible, image-based decoding solutions motivates the exploration of alternative approaches capable of handling geometric variability and uncontrolled imaging conditions.

1.3 Problem Statement

1.3.1 Core Research Problem

The core problem addressed in this thesis is the reliable decoding of Data Matrix Codes marked on industrial metal components under real-world conditions. In particular, the focus is on dot-peen marked DMCs, which remain difficult to decode using both traditional scanners and existing learning-based approaches.

The goal is to develop a decoding pipeline capable of handling variations in illumination, surface reflectance, geometric distortion, and marking irregularities, while remaining compatible with practical industrial deployment scenarios.

1.3.2 Design Constraints

The proposed solution must operate under constraints typical of industrial applications, including limited availability of annotated training data, variability in image quality, and the need for predictable and interpretable failure modes. While real-time performance is desirable, robustness and decoding reliability are prioritized in this work.

1.4 Objectives and Scope

1.4.1 Main Objective

The main objective of this thesis is to design and evaluate a robust Data Matrix Code decoding pipeline that combines learning-based detection with classical geometric

methods to improve decoding performance on industrial DMCs.

1.4.2 Sub-objectives

The specific sub-objectives of this work include:

- Learning-based localization of DMC regions in complex industrial images.
- Reliable detection of dot candidates under varying lighting and surface conditions.
- Geometric rectification and grid recovery based on detected dot patterns.
- End-to-end evaluation of decoding performance and failure modes.

1.4.3 Scope and Assumptions

This thesis focuses on still-image decoding of Data Matrix Codes and does not address continuous video-based decoding or deployment on embedded mobile hardware. The decoding process assumes compliance with the ECC200 standard and does not consider alternative bar-code symbology. The primary focus is on dot-peen marked codes, as these remain significantly more challenging for image-based decoding compared to laser-marked symbols, which are generally more accessible to existing scanners.

Experimental evaluation is conducted on small, representative symbol sizes, specifically 14×14 and 16×16 modules, reflecting common industrial use cases while maintaining a controlled scope. Conclusions drawn in this work are therefore limited to these conditions and symbol characteristics.

1.5 Research Questions

This thesis addresses the following main research question:

RQ1: How can Data Matrix Codes directly marked on industrial metal components be robustly decoded from images captured under uncontrolled industrial conditions?

This main question is investigated through the following sub-questions:

- **RQ1.1:** Can learning-based segmentation reliably localize Data Matrix Codes in industrial imagery with limited annotated data?
- **RQ1.2:** To what extent can geometric rectification and grid reconstruction compensate for irregular dot-peen markings and surface artifacts?
- **RQ1.3:** How do different pipeline stages contribute to decoding success and failure under varying image quality conditions?

1.6 Contributions

The main contributions of this thesis are:

- A systematic analysis of failure modes in traditional and learning-based Data Matrix Code decoding pipelines under industrial conditions.
- The design and implementation of a hybrid decoding pipeline that combines learning-based localization with explicit geometric reasoning to recover the underlying grid structure of dot-peen marked Data Matrix Codes.
- A dot-level decoding and rectification approach tailored to the irregular geometry and variability of dot-peen markings.
- The development of a synthetic data generation pipeline to attempt supplementation on the limited real-world training data.
- An experimental evaluation of decoding performance on industrial datasets containing dot-peen marked Data Matrix Codes, using representative small symbol sizes.

1.7 Thesis Structure

The remainder of this thesis is organized as follows. Chapter 2 reviews background theory and related work. Chapter 3 describes the proposed methodology and decoding pipeline. Chapter 4 presents experimental results and evaluations. Chapter 5 discusses limitations and future work, and Chapter 6 concludes the thesis.

2 Background Theory and Related Work

2.1 Image-Based DMC Decoding

The ISO/IEC 16022 (ECC200) specification defines the structural properties of Data Matrix Codes and the conditions required for successful decoding. From an image-based perspective, decoding requires determination of the symbol orientation and size, reconstruction of the module grid, and correct assignment of individual modules to their logical positions within the symbol. The standard further specifies the role of finder patterns, timing patterns, and the quiet zone in supporting reliable symbol detection and alignment [1].

While the specification does not prescribe a particular image processing algorithm, it implicitly assumes that the symbol can be separated from its background and that a regular sampling grid consistent with the symbol structure can be established. Once the grid has been recovered, module values can be assembled into a bitstream and decoded using the ECC200 error correction scheme.

In practical imaging scenarios, deviations from the idealized symbol appearance assumed by the standard can complicate this process. In particular, irregular module geometry or ambiguity in grid reconstruction may hinder reliable module sampling, even when the encoded symbol remains structurally valid under the ECC200 specification.

2.2 Related Prior Work

2.2.1 Industrial DMC Marking and Engraving Challenges

In industrial manufacturing, Data Matrix Codes are commonly applied directly to metal components using permanent marking techniques. An industry-focused overview by Richter [3] describes how these marking methods enable durable and permanent

identification, with Data Matrix Codes remaining readable even under partial damage due to built-in redundancy.

The article further reports that the visual appearance of engraved Data Matrix Codes is influenced by factors such as tool geometry, marking depth, spindle speed, material properties, and surface curvature. In particular, dot-peen marking can produce non-uniform cell shapes and irregular spacing when applied to curved or uneven surfaces. Such geometric distortions affect the consistency of the grid structure and, in combination with illumination conditions, complicate readability in subsequent image-based inspections, often leading to decoding failures—particularly when codes are captured outside controlled inspection stations [3, 4].

2.2.2 Dot-Based Pattern Recognition in Non-Industrial Domains

Dot-based pattern recognition has been extensively studied in the context of optical Braille character recognition, where raised dot patterns arranged on a regular grid are detected and decoded. Early work by Falcón et al. presented a Braille recognition system based on image preprocessing, grayscale conversion, dynamic thresholding, and adaptive Braille grid construction [5]. Their method included grid-based dot recovery, where valid dot positions were inferred from the grid structure to recover missed detections. High recognition accuracy was reported; however, images were acquired using flatbed scanners under controlled lighting and resolution conditions.

More recent work by Smelyakov et al. employed artificial neural networks for optical Braille character recognition, following a structured pipeline consisting of image acquisition, preprocessing, segmentation, dot recognition, and character translation [6]. Their results demonstrated high recognition accuracy on scanned Braille images, with robustness dependent on reliable dot segmentation and grid alignment.

Meng et al. proposed a two-stage Braille recognition framework based on Gaussian diffusion and semantic segmentation for Braille dot detection, followed by grid construction using traditional post-processing algorithms [7]. Their approach achieved high dot detection and character recognition accuracy on double-sided Braille datasets. Despite methodological differences, these Braille-based approaches share common assumptions of relatively controlled imaging conditions and are not designed to address surface reflectance, uncontrolled illumination, or geometric irregularities typical of industrially marked components.

2.2.3 Learning-Based DMC Decoding in Industrial Settings

Recent research has explored learning-based methods to improve the decoding of industrial Data Matrix Codes captured under non-ideal conditions. Liao et al. proposed a data extraction method for laser-marked Data Matrix Codes on metal surfaces imaged using mobile phone cameras [8]. Their approach employed convolutional neural networks to classify pairs of adjacent modules rather than individual modules, transforming the classification results into an edge image. A graph-based correction algorithm was then applied to reconstruct the Data Matrix symbol prior to decoding. Experimental results reported a substantial improvement in decoding success compared to traditional decoding software for laser-marked codes, particularly under varying illumination conditions. However, the method assumes relatively regular module geometry and was not designed to address the irregular dot structure characteristic of dot-peen marked symbols.

The most closely related prior work is the Master's thesis by Stocks [9], which investigated learning-based decoding of dot-peen marked Data Matrix Codes in industrial environments. In that work, a CNN-based object detection model was first used to locate and crop the code region from the input image. The cropped image was then processed by a neural network that produced a dense response map indicating likely dot-peen locations. From this response map, individual dot candidates were extracted, and additional dot locations were inferred by sampling and template-based matching to compensate for missing or weak detections. The resulting set of dot candidates was then used to estimate a regular Data Matrix grid, allowing the symbol to be reconstructed and passed to a standard ECC200 decoder. Experimental results showed that this pipeline could successfully decode some dot-peen marked symbols and demonstrated that learning-based processing can significantly improve robustness compared to purely traditional methods. Nevertheless, decoding performance remained limited. The reported failure cases were primarily caused by inconsistent dot geometry, missing or spurious dot detections, and error accumulation across pipeline stages. In particular, errors in early stages propagated to later grid estimation, preventing reliable reconstruction of the symbol in many cases. These limitations highlight that while learning-based methods make dot-peen decoding feasible, additional robustness and structural constraints are required for consistent performance in uncontrolled industrial settings.

2.3 Background Theory

2.3.1 Object Detection with CNNs

Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are a class of machine learning models designed for processing image data. Unlike traditional image processing methods, which rely on manually defined features, CNNs learn relevant visual patterns directly from training data. This is achieved through convolution operations, where small filters are applied across the image to detect local structures such as edges, corners, and textures.

Early layers in a CNN typically respond to simple patterns, while deeper layers combine these responses into more abstract representations. This hierarchical feature extraction allows CNNs to recognize visual objects despite variations in lighting, background, and surface appearance. As a result, CNNs are well suited for tasks involving visual detection and segmentation in complex industrial environments [10, 11].

CNNs for DMC detection

In object detection tasks, CNN-based models are trained to identify and localize objects of interest by predicting bounding boxes within an image. This approach is well suited for identifying the approximate location of a Data Matrix Code in cluttered industrial scenes, as it allows subsequent processing stages to focus on a reduced region of interest. Object detection has therefore been adopted in prior work as an effective first step for isolating DMCs from complex backgrounds.

In the work by Stocks [9], a CNN-based object detection model was used to locate and crop the Data Matrix Code region prior to further processing. While this step improved robustness compared to traditional heuristic-based localization, object detection alone was not sufficient to guarantee reliable decoding. Bounding boxes may be loose or imperfectly aligned, and rotations of the symbol within the detected region can introduce additional geometric distortion. As a result, accurate detection must be followed by further processing stages that explicitly recover the internal grid structure of the symbol.

2.3.2 Contrast Normalization

Contrast normalization refers to a class of image preprocessing techniques that aim to reduce the influence of non-uniform illumination and surface reflectance on image appearance. In industrial imaging, particularly on metallic surfaces, variations in lighting, glare, and surface finish can cause the visual contrast of Data Matrix Code features to vary significantly across the image. Such variations can hinder subsequent processing stages that rely on consistent intensity differences between foreground and background.

Prior work commonly applies contrast normalization as an early preprocessing step to stabilize the visual appearance of code features. In the context of learning-based pipelines, this is typically achieved through grayscale normalization or illumination flattening techniques that suppress large-scale intensity gradients while preserving local structure. Stocks [9], for example, applies illumination normalization prior to neural network inference in order to reduce sensitivity to uneven lighting on dot-peened surfaces. Similar preprocessing is also reported in related work on laser-marked codes, where contrast enhancement improves the reliability of downstream detection and reconstruction [8].

2.3.3 Blob Extraction and Dot Candidate Generation

Blob extraction refers to the process of identifying localized regions in an image that correspond to distinct visual structures, such as individual dot-peened marks. In the context of Data Matrix decoding, blob extraction is used to generate candidate dot locations that may correspond to symbol modules. These candidates serve as the basis for subsequent geometric reasoning and grid reconstruction.

In prior work, blob extraction is typically performed after preprocessing or neural network inference, using image responses that highlight likely dot locations. Stocks [9], for instance, extracts dot candidates from a neural network response map by identifying connected regions and filtering them based on size and shape. Similar connected-component and blob-based approaches are widely used in dot-based pattern recognition, including Braille recognition systems, where individual dots are detected prior to grid construction [5]. While blob extraction can produce false or missing detections in challenging conditions, it provides a practical mechanism for reducing dense image data to a sparse set of geometrically meaningful candidates.

3 Proposed Decoding Pipeline

3.1 Overview of the Proposed Pipeline

Building on the limitations of traditional Data Matrix Code (DMC) decoding approaches identified in Chapter 2, this work proposes a modular decoding pipeline specifically designed for challenging industrial imagery, with a particular focus on dot-peen marked symbols. Rather than attempting to directly decode raw images under adverse conditions, the proposed pipeline reconstructs a decoder-friendly representation of the symbol by combining learning-based detection with geometry-driven reasoning.

At a high level, the pipeline processes an input image through a sequence of stages that progressively transform unstructured visual data into a clean, synthetic representation compatible with standard ECC200 decoders. The overall flow of the system can be summarized as follows:

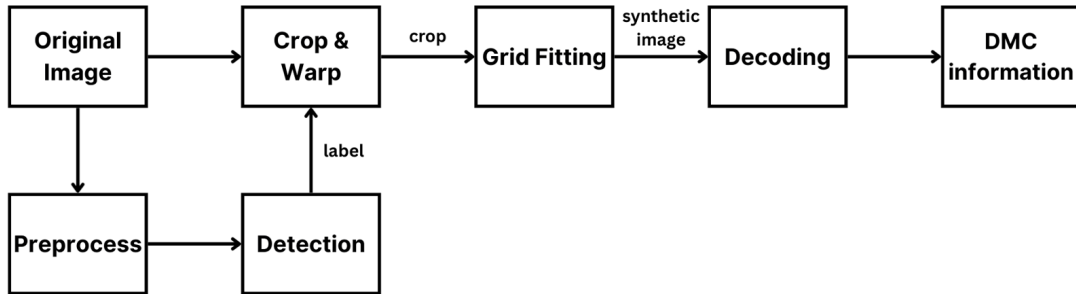


Figure 3.1: Pipeline processing workflow

The proposed decoding pipeline is illustrated in Figure 3.1. Starting from the original input image, the first step consists of detecting the location of the Data Matrix Code within the scene using a learning-based segmentation and detection approach. This step isolates the symbol from its surrounding environment and provides a geometric description of its position.

Using the predicted detection, the code region is then cropped from the original image and warped into a fixed square reference frame. This rectification step removes perspective distortions and ensures that subsequent processing operates on a normalized symbol representation, independent of its original orientation or scale.

Once rectified, the symbol is classified according to its marking type. For laser-marked symbols, a simple contrast enhancement and quiet-zone padding are sufficient before decoding. For dot-peen symbols, additional processing is required: individual dots are detected and treated as geometric primitives, which are then used to estimate the underlying Data Matrix grid structure.

Based on the recovered grid, a clean synthetic binary image of the symbol is constructed. This reconstructed image explicitly enforces the expected Data Matrix layout, including finder patterns, timing patterns, and quiet zones. The final synthetic image is then passed to a standard ECC200 decoder to recover the encoded information.

3.2 Data Acquisition and Datasets

The dataset used in this work was initially created in the context of a previous Master's thesis by Stocks [9], which addressed the same industrial decoding problem. In that project, images were collected by Everllence staff and collaborators under real production conditions and used to explore learning-based decoding approaches.

Following this initial phase, the dataset was expanded in two subsequent stages. In the first expansion, additional images were provided, manually annotated and incorporated into the training and evaluation pipeline used in this work. In a later phase, a separate set of images was collected with the intention of being used exclusively for testing. However, a large portion of this material was found to be out of scope due to unsupported symbol sizes, severe marking defects, low image quality, or a predominance of laser-marked symbols. After filtering, the remaining images were retained and included in the final dataset, which is not included in the analysis that follows.

3.2.1 Marking Technique

The real dataset used in this work consists of industrial images containing Data Matrix Codes marked directly on metal components. Both dot-peen and laser-marked symbols are present in comparable numbers, as shown in Figure 3.2. This balance ensures that the proposed pipeline must reliably handle both marking technologies rather than being optimized for a single case.

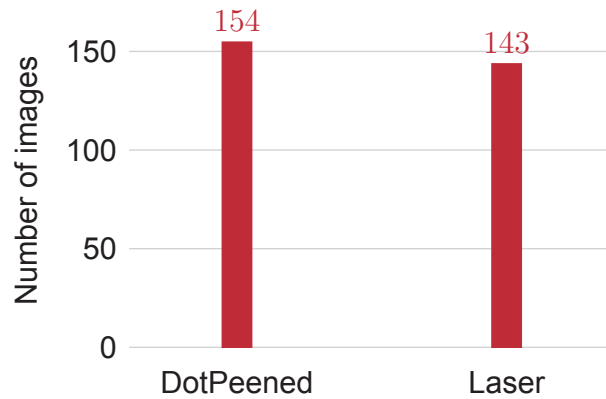


Figure 3.2: Distribution of marking types in the real dataset.

3.2.2 Code-to-Background Polarity

In addition to differences in marking technique, the dataset exhibits significant variability in grayscale polarity. As illustrated in Figure 3.3, both dark-on-light and light-on-dark symbols occur frequently. Furthermore, glare artifacts are present in both polarity configurations, indicating that neither polarity inversion nor simple contrast assumptions can be applied globally across the dataset.

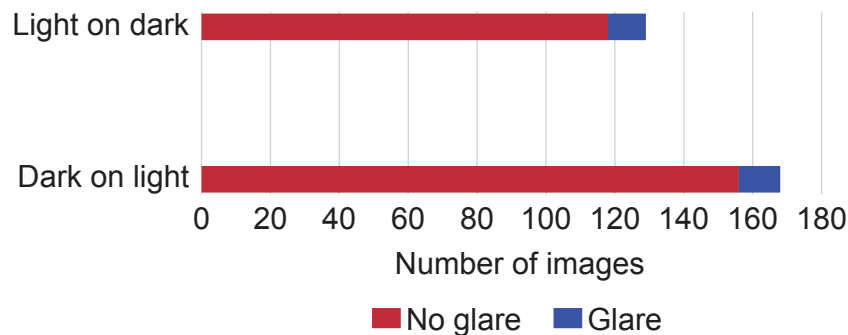


Figure 3.3: Grayscale polarity distribution and presence of glare artifacts.

3.2.3 DMC Standards Violation

Beyond visual degradations, the dataset also contains structural violations of the Data Matrix standard. In several images, the required quiet zone surrounding the symbol is partially or completely absent, with the code directly adjacent to other markings, edges, or textured surface regions. Such violations prevent reliable decoding by standard ECC200 decoders, which assume the presence of a sufficient quiet zone, as discussed in Section 1.1. These cases introduce ambiguity at the symbol boundary and further complicate reliable grid recovery.

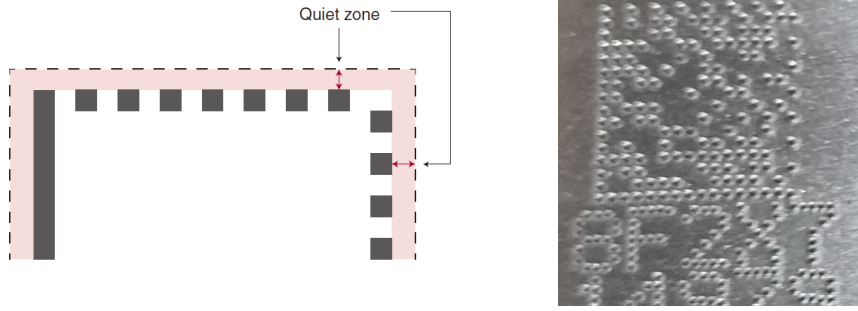


Figure 3.4: Example image with no-quiet-zone violation.

3.2.4 Code-to-Image Area Quality

Another important characteristic of the dataset is the relative size of the Data Matrix Code within the image. Figure 3.5 shows the distribution of the code-to-image area ratio. The majority of images contain symbols occupying less than 5% of the total image area. In these low-ratio cases, the code is represented by a limited number of pixels, making decoding particularly sensitive to blur, noise, and surface reflections.

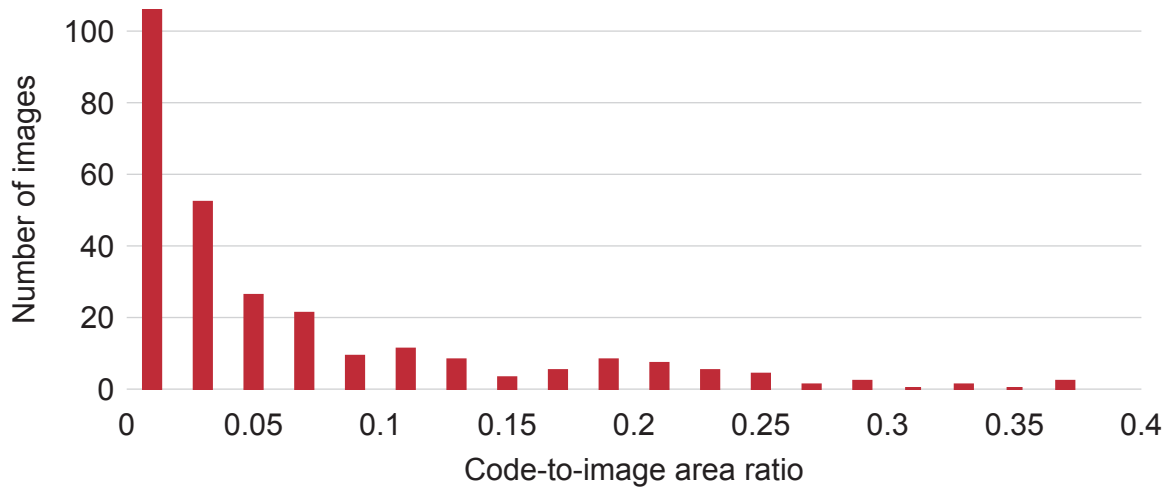


Figure 3.5: Distribution of the code-to-image area ratio in the real dataset.

To further characterize image quality, a threshold of 5% is used to distinguish between low-quality and high-quality samples based on the code-to-image area ratio. Using this criterion, a substantial portion of the dataset falls into the low-quality category, where reliable decoding using traditional methods is challenging. This observation strongly motivates the use of a learning-based detection stage followed by geometric rectification before decoding.

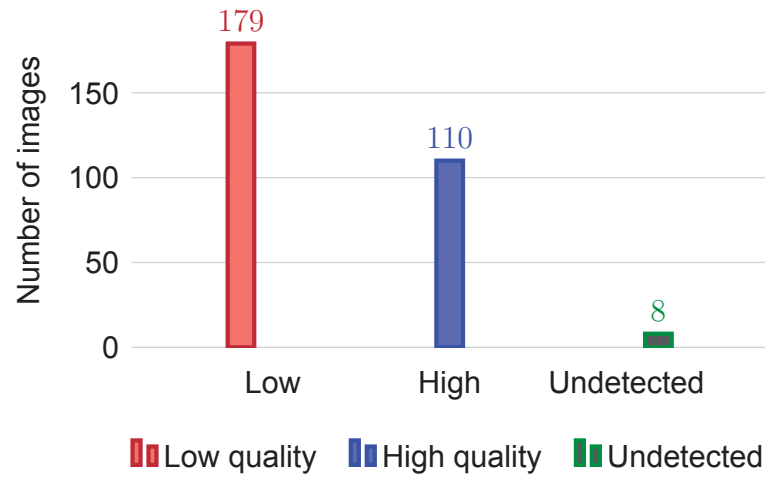


Figure 3.6: Distribution of image quality based on the code-to-image area ratio using a threshold of 5%. Images labeled as undetected were not localized by the detection model.

Finally, image resolutions vary widely across the dataset, ranging from low-resolution captures to high-resolution industrial imagery. This variability precludes the use of fixed pixel-scale assumptions and requires subsequent processing stages to adapt automatically to the image scale.

To address the limited availability of annotated real data and to expose the pipeline to controlled variations that are rare in practice, the real dataset is complemented with synthetically generated images. The synthetic data generation process is described in detail in Section 3.3.

3.3 Synthetic Data Generation

Due to the limited availability of annotated industrial images and the large variability observed in the real dataset, a synthetic data generation pipeline was developed to explore data augmentation strategies and controlled training scenarios. The goal of this effort was to investigate whether synthetic samples could improve detection robustness under challenging conditions.

Synthetic Data Matrix Codes are generated by explicitly constructing valid Data Matrix grid layouts according to ECC200 standards and rendering them as dot-peen symbols. Rather than drawing idealized circular dots, the generation process uses real dot patches extracted from industrial images, which are randomly rotated, scaled,

and intensity-jittered before being placed on the grid. This patch-based approach produces more realistic dot shapes and local texture variations than purely procedural rendering.

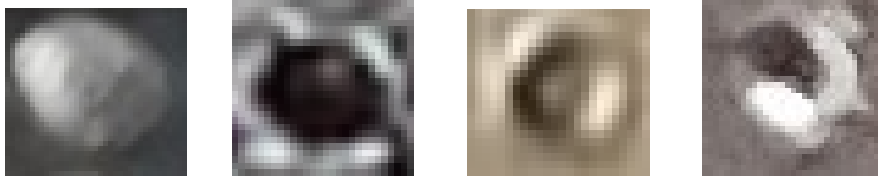


Figure 3.7: Examples of dot patches used as samples.

To simulate realistic industrial backgrounds, synthetic symbols are embedded into metal-like textures generated procedurally or sampled from real surface images. Additional degradations, including blur, noise, scratches, glare streaks, and mild perspective distortions, are applied to reflect common imaging artifacts observed in practice. The scale and placement of the symbol within the image are varied to reproduce the wide range of code-to-image area ratios observed in the real dataset.

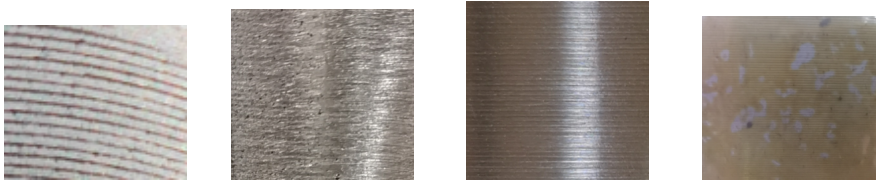


Figure 3.8: Examples of metallic surfaces used as backgrounds.

For each synthetic image, exact geometric annotations of the Data Matrix Code are generated automatically, including oriented bounding boxes and known grid sizes.

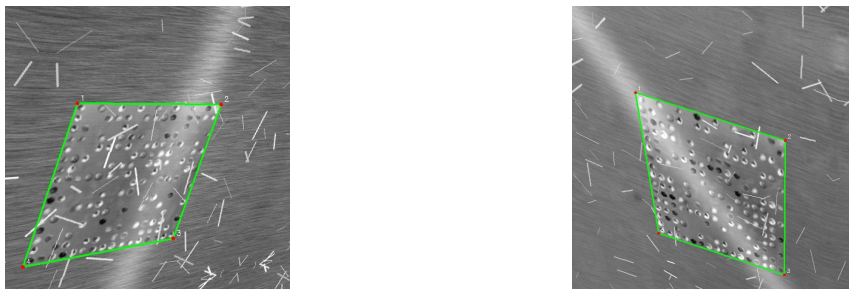


Figure 3.9: Examples of final synthetic images with glare, scratches and warping. The green boxes surrounding are for validating the bounding boxes.

While the synthetic data pipeline provides precise geometric annotations and controlled variability, empirical evaluation indicated that models trained primarily on real

industrial images achieved more consistent detection performance on the target dataset. As a result, synthetic data was not used in the final training configuration of the detection model evaluated in Section 4.2.

3.4 Preprocessing and Contrast Normalization

3.4.1 Role of preprocessing in the pipeline

All images used in this work are preprocessed prior to both training and inference. This includes real industrial images as well as synthetically generated samples. The purpose of preprocessing is to reduce low-frequency illumination variations and glare effects that were identified as dominant sources of variability in Section 3.2.

By applying the same preprocessing procedure to all data, the learning-based detection model operates on a consistent input distribution. This avoids mismatches between training and deployment and ensures that the detector learns to focus on structural characteristics of the Data Matrix Code rather than global brightness or surface-dependent artifacts.

3.4.2 Gaussian-based background estimation

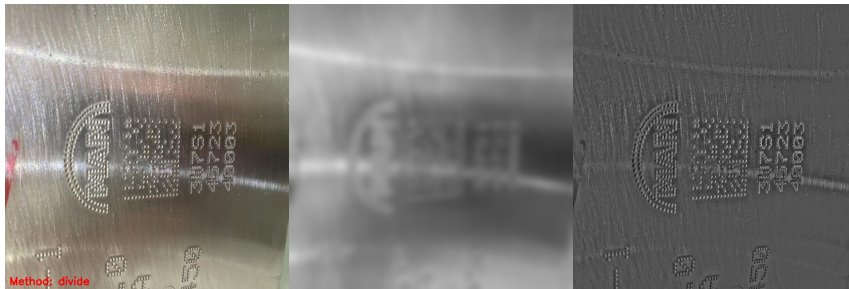
Preprocessing is based on Gaussian filtering to estimate the background illumination of the image. Given a grayscale image, a Gaussian blur with a large kernel size is applied to produce a smoothed version that captures low-frequency intensity variations while suppressing fine structural details such as individual dot-peen marks [12].

This operation is implemented using the `GaussianBlur` function from the OpenCV library [13]. The blurred image serves as an approximation of the background illumination, which can then be removed from the original image to enhance local contrast and suppress glare.

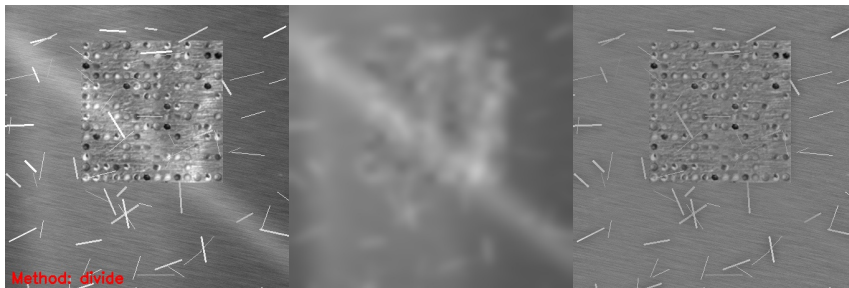
3.4.3 Parameter selection and normalization strategy

In this work, the Gaussian kernel size is chosen to be significantly larger than the expected size of individual dot-peen marks. This ensures that local dot structure is suppressed in the background estimate while global illumination trends and surface reflectance variations are preserved.

After background estimation, the original image is normalized using pointwise division by the blurred image. This operation compensates for multiplicative illumination effects and variations in surface reflectivity, which are common in metallic industrial components. Compared to additive normalization strategies, division preserves local contrast in a scale-invariant manner and reduces sensitivity to absolute brightness levels, making it well suited for the imaging conditions observed in the dataset. The resulting normalized image is used directly as input to the detection model, without further binarization or thresholding.



(a) Real image preprocessing.



(b) Synthetic image preprocessing.

Figure 3.10: Effect of Gaussian-based preprocessing, which removes background illumination from the original image to enhance local contrast.

3.4.4 Application during training and inference

The preprocessing procedure described above is applied uniformly to all images used for training, validation, and inference. Synthetic images are preprocessed using the same pipeline as real images, ensuring that the detection model is not exposed to discrepancies between simulated and real data.

During inference, each new input image is first preprocessed before being passed to the learning-based detection model. This guarantees that the detector operates under the same conditions as during training and allows subsequent pipeline stages to rely on consistent contrast characteristics.

3.5 Learning-Based DMC Detection and Segmentation

3.5.1 Introduction and role in the pipeline

The detection stage is responsible for locating the Data Matrix Code within the input image and providing a precise geometric description of its spatial extent. As discussed in Section 2.3, this step isolates the symbol from its surrounding environment and serves as the entry point to all subsequent processing stages in the proposed pipeline. Accurate detection is particularly critical for low-quality images, where the code occupies only a small fraction of the image area, as discussed in Section 3.2. The detection model operates exclusively on preprocessed images, as described in Section 3.4, ensuring consistent input statistics during training and inference.

3.5.2 Model choice: YOLO segmentation

Detection is performed using a YOLO-based segmentation model. Compared to traditional object detection pipelines that rely on multi-stage processing (e.g. region proposal followed by classification), YOLO adopts a single-stage, fully convolutional formulation in which localization and classification are performed jointly in a single forward pass. This design significantly reduces computational overhead and latency, making the approach well suited for large image batches and time-constrained industrial workflows.

From a technical perspective, YOLO predicts object locations directly from dense feature maps produced by a backbone network, avoiding the need for explicit region proposal mechanisms. By operating on multi-scale feature representations, the model can detect objects across a wide range of spatial resolutions, which is particularly important in this work, as many Data Matrix Codes occupy only a small fraction of the image area.

In addition to efficiency, YOLO provides a robust framework for learning spatial context. The use of shared convolutional features allows the detector to exploit global image information when localizing the symbol, improving robustness in cluttered scenes and under uneven illumination. These properties make YOLO substantially more effective than conventional heuristic-based or sliding-window approaches for Data Matrix Code localization in industrial imagery.

In this work, a YOLO segmentation variant is used, which is predicting a pixel-level

mask for each detected instance. YOLO-style instance segmentation augments object detection with a prototype-based mask representation. A segmentation head predicts a fixed set of spatial prototype masks shared across the image, while each detected object predicts a corresponding vector of mask coefficients. The final instance mask is obtained as a linear combination of the prototype masks weighted by the object-specific coefficients, followed by cropping to the predicted bounding box. For better understanding, very high-level conceptual visualizations of the YOLO object detection and segmentation processes can be found in the B.1 of the Appendix. The model is open-source and freely available for research use, enabling reproducible experimentation and straightforward integration into the proposed pipeline.

Segmentation Masks

Unlike standard object detection, which outputs axis-aligned or slightly rotated bounding boxes, segmentation-based detection provides a dense spatial mask of the symbol region. This distinction is particularly important for industrial Data Matrix Codes, which may appear rotated, partially occluded, or tightly surrounded by other structures.

Using segmentation masks enables tighter and more accurate crops of the symbol region. This reduces background interference during the subsequent geometric rectification step and improves robustness when the symbol is rotated or viewed under perspective distortion. This segmentation mask come in the for a surrounding contour, which is easy to manipulate geometrically, without further image processing. In contrast, loose bounding boxes may include irrelevant background regions, leading to increased error propagation in later stages.

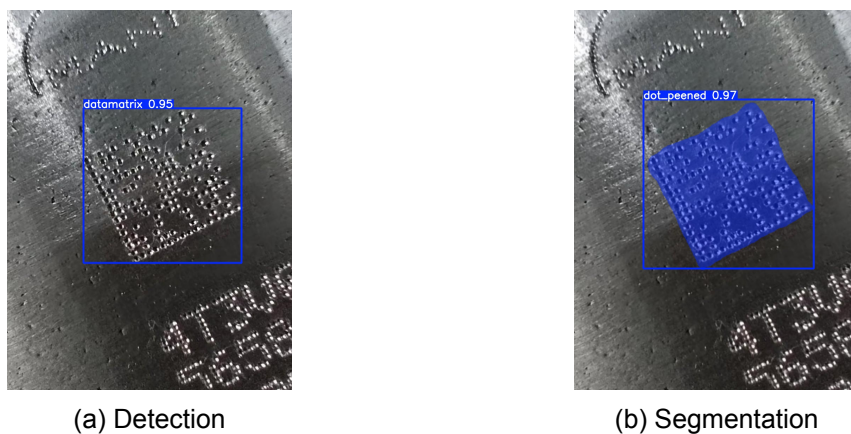


Figure 3.11: Object Detection result compared to Segmentation result

3.5.3 Training datasets and configurations

Multiple detection models were trained using different combinations of real and synthetic training data. The configurations varied in model sizes, the proportion of synthetic samples, the inclusion of inverted images, and data augmentation settings. This allowed an empirical evaluation of how training data composition affects detection performance on challenging industrial imagery. Details about the results can be found in B of the Appendix.

3.5.4 Model selection and justification

The trained models were evaluated based on their ability to correctly detect Data Matrix Codes in the real dataset. Performance was measured in terms of successful detections.

Based on this evaluation, the final selected model corresponds to the YOLOv11-nano segmentation variant trained exclusively on real images with augmentation, which provided the most consistent detection performance on the real-image test set. The model's training specifications and results can be found in the Appendix's B.2 and B.8 respectively. This model is used in all subsequent experiments presented in this thesis.

3.6 Rectification from Segmentation Masks

After the detector localizes a Data Matrix Code, the next step is to extract the symbol region from the original image and rectify it into a canonical, square view. This step reduces background clutter, compensates for rotation and perspective distortion, and produces a normalized crop that is compatible with the downstream grid fitting stage. An overview of the intermediate results is shown in Figure 3.12, where each panel corresponds to one of the processing steps implemented.

The detection model outputs a segmentation polygon in normalized image coordinates. For each image, the polygon with the highest confidence score is selected and transformed into pixel coordinates. The subsequent processing then converts this raw mask into an ordered set of four corners that define a projective transformation used for warping the symbol region to a fixed resolution. All the methods used for

the rectification are from the OpenCV library [13] and all the stages explained below are visualized in the following figure.

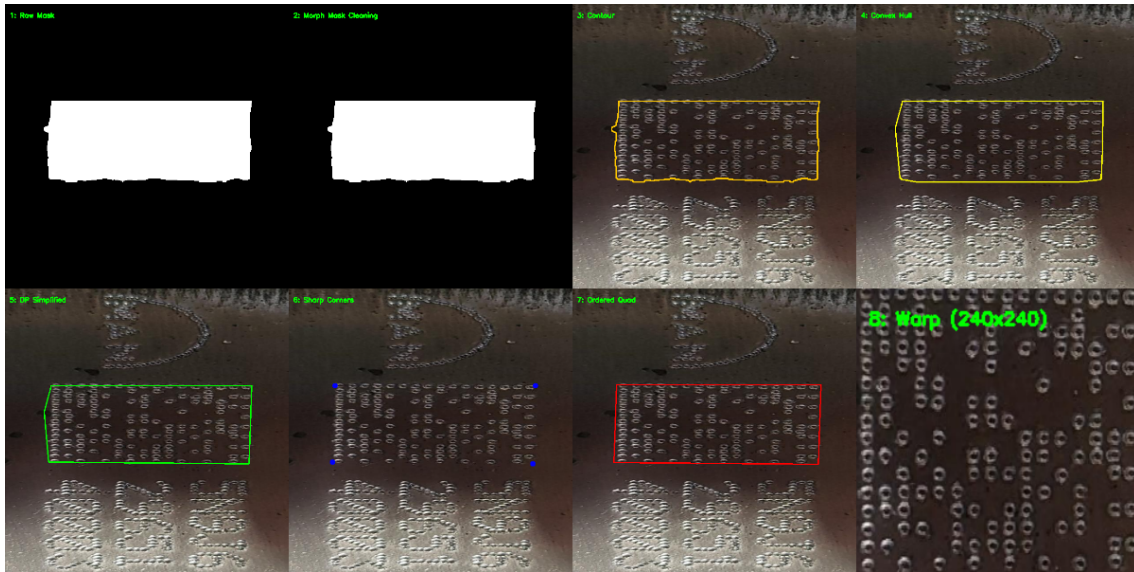


Figure 3.12: Debug visualization of the crop-and-warp procedure used to rectify the detected symbol region. The panels correspond to: (1) raw mask, (2) morphological mask cleaning, (3) contour extraction, (4) convex hull, (5) Douglas–Peucker simplification, (6) corner selection, (7) ordered quadrilateral, and (8) final warped crop.

3.6.1 Raw mask generation

The predicted segmentation polygon is rasterized into a binary mask by filling the polygon region. This produces a pixel-level representation of the detected symbol area and provides a robust starting point for geometric processing, even when the predicted contour is slightly irregular.

3.6.2 Morphological mask cleaning

The raw binary mask produced from the segmentation polygon may contain thin spurious structures attached to the main symbol region. These artifacts typically arise from segmentation uncertainty and appear as narrow bridges or small protrusions connecting the Data Matrix region to unrelated background pixels. If left uncorrected, such structures distort the extracted contour and can introduce false corners.

To address this issue, a morphological cleaning stage is applied directly to the binary mask. First, a morphological opening is performed, consisting of an erosion followed by a dilation using a 3×3 structuring element [14]. This operation effectively removes

thin connections and narrow bridges while preserving the main symbol region, as such artifacts are usually only a few pixels wide.

After opening, connected component analysis is used to retain only the largest connected foreground region, under the assumption that the Data Matrix occupies the dominant area of the mask. This step removes any remaining isolated clutter or detached fragments produced by the segmentation model.

Finally, a light morphological closing is applied to the cleaned mask to restore small gaps or boundary discontinuities introduced by the opening operation. The result is a single, well-connected mask that accurately represents the symbol region while suppressing spurious attachments and segmentation noise. Visual explanation of the morphological processing can be found in the C.1 of the Appendix.

3.6.3 Contour extraction

From the closed binary mask, contours are extracted to obtain an explicit boundary representation of the detected region. Since the segmentation mask may contain small spurious regions or holes, only the largest contour by area is retained. This contour represents the outer boundary of the detected Data Matrix Code and serves as the basis for subsequent geometric processing.

3.6.4 Convex hull

The extracted contour may still contain concavities caused by segmentation noise or partial occlusions. To obtain a stable geometric envelope, the convex hull of the contour is computed [15]. The convex hull removes inward boundary deviations and produces a shape consistent with a planar quadrilateral observed under perspective projection. A visualization of this step can be found in C.6.

3.6.5 Douglas-Peucker simplification

Although the convex hull provides a clean envelope, it typically consists of many closely spaced points along its boundary. To reduce sensitivity to small boundary variations and to obtain a compact polygonal representation, the hull is simplified using the Douglas–Peucker algorithm [16].

Simplification is performed using a tolerance parameter ϵ is set to 1% of the hull

perimeter. This choice ensures scale-invariant behavior: points that lie close to a straight boundary segment are removed, while geometrically meaningful corner points are preserved. As a result, the simplified polygon retains the dominant shape of the detected region while eliminating redundant vertices. An example result of using this method can be found in the Appendix's C.2.

3.6.6 Corner selection

After simplification, the resulting polygon may still contain more than four points. Since the Data Matrix region is expected to be a quadrilateral under perspective projection, four corners are selected by choosing the four vertices with the smallest internal angles (the sharpest corners). This provides a stable estimate of the symbol corners even when the polygon contains extra vertices.

3.6.7 Corner ordering

To compute a valid projective transform, the four selected corners must be ordered consistently. The corners are ordered as top-left, top-right, bottom-right, and bottom-left using a centroid-based angular sorting strategy.

First, the centroid of the four corner points is computed. Each corner is then assigned an angle relative to this centroid, and the points are sorted in counter-clockwise order based on these angles. This produces a stable cyclic ordering of the corners that is independent of the symbol orientation in the image.

After ordering, the quadrilateral is expanded outward by a small padding factor to ensure that the full symbol region is retained during cropping. The expansion is performed radially from the quadrilateral centroid by scaling the corner vectors outward. The padding is applied relative to the quadrilateral size and differs by marking type: a padding of 2% is used for dot-peen symbols, while a larger padding of 5% is applied for laser-marked symbols to account for engraved boundary variability.

3.6.8 Perspective warp

Finally, the ordered corners define a homography that maps the detected symbol region into a square image of fixed size (240×240 pixels). The warp is computed using a perspective transformation, producing a rectified crop in which rotation and perspective distortion are removed. Homography-based rectification is a standard

approach in computer vision [17].

Nearest-neighbor interpolation is used during warping to preserve the sharp local structure of dot-peen marks and avoid smoothing artifacts; something that can be introduced by bilinear interpolation.

3.7 Geometric Reasoning and Grid Fitting

This stage recovers the discrete grid structure of the Data Matrix Code from the rectified crop. Unlike the learning-based detection stage, grid fitting relies entirely on geometric reasoning and structural constraints defined by the Data Matrix standard mentioned in 1.1.

The objective of grid fitting is to estimate a regular two-dimensional lattice that aligns with the physical dot structure of the symbol, despite missing, spurious, or distorted dot detections. This allows the symbol to be reconstructed in a normalized, synthetic form that is suitable for standard ECC200 decoding.

3.7.1 Laser-marked symbol processing

After detection and rectification, the pipeline branches depending on the marking technology. Laser-marked Data Matrix Codes are processed using a dedicated pre-processing path that differs from the dot-peen branch described in the following sections.

Laser-marked symbols typically exhibit continuous module regions with sharper boundaries and more regular geometry compared to dot-peen marks. However, they are still affected by non-uniform illumination, surface reflectivity, and local contrast variations, particularly on metallic substrates. These effects can reduce the contrast between modules and background, leading to decoding failures when raw images are passed directly to standard decoders.

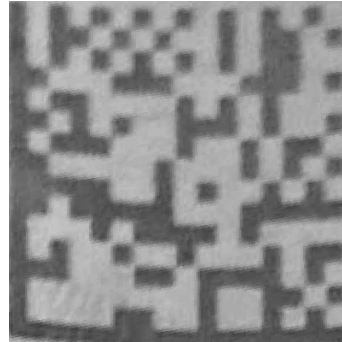
To address these issues, laser-marked crops undergo contrast enhancement prior to decoding using Contrast Limited Adaptive Histogram Equalization (CLAHE) [13]. CLAHE operates on local image regions and redistributes intensity values to enhance local contrast while limiting noise amplification. This is particularly effective for laser-marked symbols, where module boundaries are already well defined but may be lo-

cally suppressed by uneven illumination or surface reflectivity.

The output of this preprocessing stage is an enhanced grayscale image in which laser-engraved modules exhibit improved contrast and uniformity. Unlike the dot-peen branch, no blob detection or geometric grid fitting is required for laser-marked symbols. The enhanced image is passed directly to the ECC200 decoder, which can reliably decode the symbol once sufficient local contrast has been restored.



(a) Laser-marked DMC crop.



(b) Laser-marked DMC after CLAHE processing.

3.7.2 Motivation for geometric reasoning

After rectification, decoding dot-peen marked Data Matrix Codes remains challenging due to variability in dot appearance, spacing, and visibility. Individual dots may be weak, partially missing, or merged with surface texture, making direct per-cell classification unreliable even under normalized geometry.

At this stage of the pipeline, the objective is therefore not to interpret local appearance in isolation, but to recover a globally consistent grid structure that is compatible with the Data Matrix specification. By enforcing geometric regularity and structural constraints, the pipeline can tolerate local detection errors while still producing a valid symbol layout. This shifts the decoding problem from appearance-based classification to structural estimation, which is more robust under the physical degradation observed in industrial dot-peen markings.

3.7.3 Polarity normalization

As discussed in Section 3.2, dot-peen symbols may exhibit either light-on-dark or dark-on-light appearance due to illumination conditions and surface reflectivity. Since the blob detection stage, discussed later, assumes bright dot responses, such polarity inversions must be corrected prior to further processing.

Rather than applying a global inversion, which can be unreliable under non-uniform illumination, polarity normalization is performed locally. The rectified crop is divided into four quadrants, and each quadrant is analyzed independently. For each quadrant, polarity is determined through grayscale histogram analysis. The grayscale intensity histogram is computed, and the dominant intensity mode is identified as the location of the highest histogram peak. This peak provides a robust estimate of whether the quadrant background is predominantly bright or dark. If the dominant mode exceeds a predefined intensity threshold, the quadrant is classified as having a light background with darker dots, and an inversion is marked as required for that quadrant.

To reduce sensitivity to local artifacts and prevents erroneous inversion caused by isolated glare or dense dot regions, the following spatial consistency constraint was applied: An inversion is applied only if at least two adjacent quadrants indicate inconsistent polarity. If a single quadrant, or two non-adjacent quadrants, exhibit inverted polarity, no inversion is performed. This strategy exploits the observation that glare effects on metallic surfaces typically extend across contiguous regions rather than isolated areas.

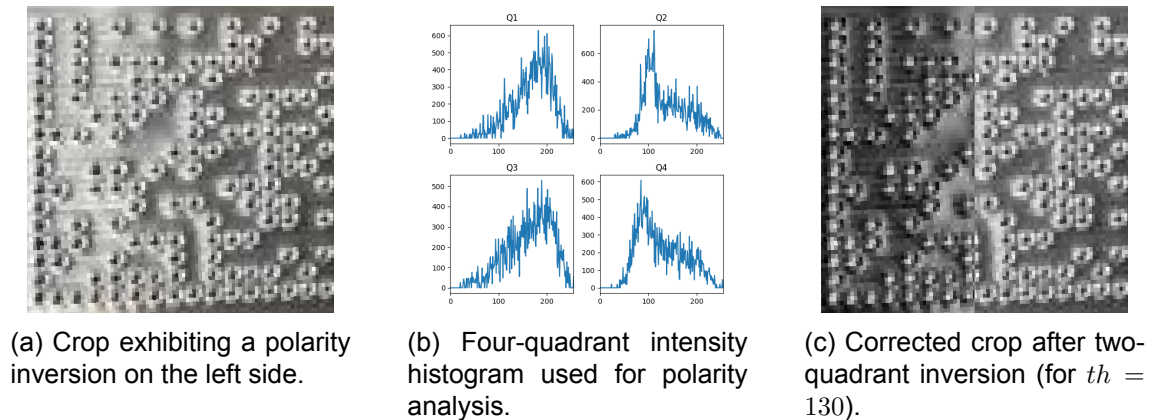


Figure 3.14: Polarity normalization process for dot-peened images with mixed illumination.

3.7.4 Blob-based dot candidate detection

Following polarity normalization, dot-peen marks are detected as localized blob responses using a Laplacian-of-Gaussian (LoG) operator [18]. The blob detection is applied to the rectified symbol crop prior to grid fitting and uses the original grayscale image rather than the blurred image used during detection preprocessing. This is because the LoG operator internally applies Gaussian smoothing at multiple scales, making additional external blurring unnecessary and potentially detrimental to dot lo-

calization accuracy.

Blob detection is performed using the `scikit-image` implementation of LoG, which identifies scale-normalized extrema corresponding to bright, approximately circular structures. The expected dot size is estimated from the current grid cell sizes that are in scope, and this estimate is used to constrain the range of evaluated scales. Specifically, the maximum scale is set based on the expected dot radius, while a lower bound is enforced to suppress detections caused by fine surface scratches, glare artifacts, or noise.

Additional filtering is applied through overlap suppression, ensuring that multiple detections do not occur for a single physical dot. The resulting blob detections are represented by their center coordinates and associated scale in pixel units. This sparse set of dot candidates forms the geometric input for subsequent grid size estimation and fitting stages.

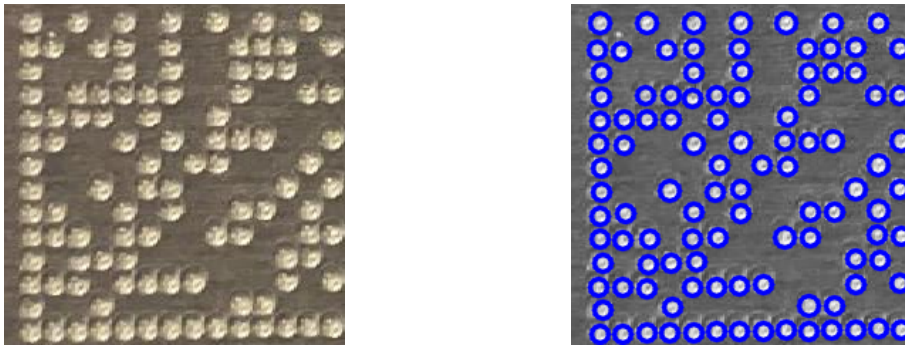


Figure 3.15: Blob detection on dot-peen marked DMC crop.

3.7.5 Border fitting

Following quadrant polarity normalization and LoG-based blob detection, the pipeline estimates the four symbol borders directly from the detected dot centers. This stage is designed to handle strong tilt, residual perspective distortion, and local artifacts (scratches and glare) without relying on an early grid-size estimate. The method assumes that, according to ECC200 standard 1.1, the Data Matrix Code boundary can be represented by four lines (top, right, bottom, left) and that valid symbols exhibit two adjacent solid finder borders (the L-pattern).

Band-based candidate selection

Border estimation is performed independently for each of the four symbol sides (top, right, bottom, and left) using an iterative band-search strategy. For a given side, blob centers are selected from a narrow strip adjacent to the image boundary. If the collected evidence is insufficient, the strip is shifted progressively inward in fixed steps, up to a predefined maximum depth. All band-related parameters are expressed relative to the estimated grid cell size, ensuring scale-invariant behavior across symbols of different resolutions. Each inward shift implicitly expands the set of eligible blob centers, such that all blobs admitted in previous bands remain available for subsequent line estimation. This overlapping band progression ensures that the true border row is not skipped while the depth limit prevents the fitting process from drifting into interior grid rows.

At each band position, only the outermost blob centers closest to the corresponding edge are used for line estimation. A straight line is fitted using a total least-squares estimator (`cv2.fitLine`), which minimizes orthogonal distances and is well suited for geometric estimation under noise, yielding a candidate border hypothesis [19]. As the search progresses, the algorithm retains the candidate line with the strongest blob support, ensuring that the final border estimate reflects the most reliable alignment observed near the symbol boundary.

Support counting with inner-core constraint

To reject incidental contacts caused by scratches and glare, a blob is considered supporting a candidate border line only if the line passes sufficiently close to the blob center. For each blob i , let r_i denote its radius estimate (derived from LoG scale). The perpendicular distance from point $\mathbf{p}_i$ to line L_s of side s is:

$$d(\mathbf{p}_i, L_s) = \frac{|a_s x_i + b_s y_i + c_s|}{\sqrt{a_s^2 + b_s^2}}. \quad (3.1)$$

A blob supports the line only if it intersects the blob inner core, defined as a fixed fraction γ of the blob radius:

$$d(\mathbf{p}_i, L_s) \leq \gamma r_i, \quad \gamma = 0.6. \quad (3.2)$$

The support count for side s is the number of blobs satisfying this criterion. Across band expansions, the algorithm maintains the *best* line per side (highest support),

which makes the procedure monotonic and robust to early noisy fits.

Two-phase locking

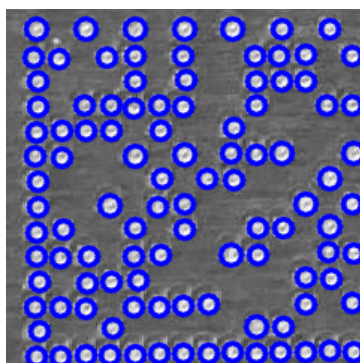
Border identification proceeds in two phases. First, the algorithm searches for two adjacent sides forming the L-pattern. Among the four adjacent side pairs

(top, left), (top, right), (bottom, left), (bottom, right),

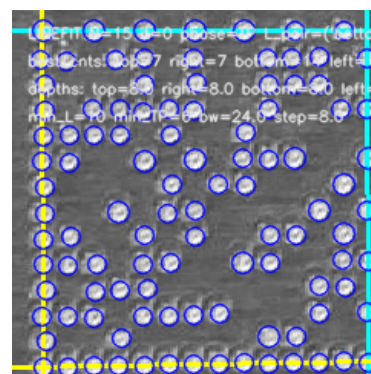
the pair with the highest combined support is selected as the L-candidate. The pair is locked only if both sides exceed a minimum support threshold (a fraction of the expected border length). Once two adjacent L borders are locked, the remaining two sides are searched and locked with a lower threshold, reflecting the fact that timing borders may be less reliably detected. The output of this stage is: (i) four locked border lines; and (ii) the identified adjacent L-pair.

Quad construction and re-rectification

To ensure that the subsequent warp encloses the full symbol (including the border dots), each locked border line is offset outward by half a cell. The cell size is estimated from the median nearest-neighbor distance (pitch) of blob centers, denoted p . Therefore, each border line is shifted by $\delta = 0.5p$ along its outward normal. The intersections of the four offset lines define a refined quadrilateral, which is warped to a square crop using the same homography-based transformation as in the initial rectification stage 3.6.8. This produces a rectified symbol crop in which the grid is approximately axis-aligned and geometric distortion is minimized.



(a) Initial blob detection.



(b) Border fitting. The yellow lines represent the finding pattern and the blue ones the timing-pattern

Figure 3.16: Border fitting stage.

3.7.6 Grid fitting

After border fitting and warping, the pipeline re-runs blob detection on the rectified crop and performs grid fitting. This is where the pipeline discriminates between candidate grid sizes using a strict L-pattern occupancy rule and collects geometric data for the finalized symbol.

Candidate sizes and evaluation order

The pipeline tests candidate grid sizes in descending order, starting with the maximum plausible symbol:

$$N \in \{16, 14\}, \quad \text{evaluated in the order } 16 \rightarrow 14.$$

This choice reflects the common occurrence of missing border evidence: it is preferable to accept a valid larger grid when possible and only fall back to a smaller size if the larger hypothesis is inconsistent.

Grid overlay and cell occupancy

Given a warped crop of size $H \times W$ and a candidate size N , a regular $N \times N$ grid is overlaid on the crop. Each blob center $\mathbf{p}_i = (x_i, y_i)$ is assigned to a grid cell and the binary occupancy grid $G \in \{0, 1\}^{N \times N}$ is set as $G_{j,i} = 1$ if at least one blob falls into that cell.

L-pattern acceptance rule

Let $L_{\text{pair}} = (s_1, s_2)$ be the two adjacent finder borders identified during border fitting and for each side s , let C_s be the number of occupied cells along that border. A candidate size N is accepted if and only if both L borders are nearly complete:

$$C_{s_1} \geq N - 1 \quad \wedge \quad C_{s_2} \geq N - 1. \quad (3.3)$$

This rule uses only the solid finder borders, as timing borders may appear loose due to partial dot loss or residual warping error. If the initial N hypothesis fails this test, the pipeline evaluates the remaining smaller ones using the same rule.

If neither $N = 16$ nor $N = 14$ satisfies the L-pattern occupancy rule, the rectified symbol is rejected to prevent unstable reconstruction. In such cases, the pipeline

exports a contrast-normalized fallback image, processed as a laser-marked DMC, to prevent mistakes that can occur when laser marked codes are wrongly classified as dot peened ones.

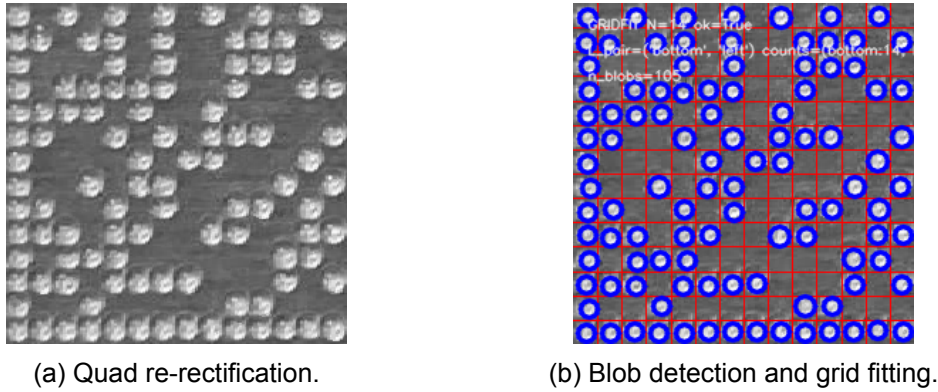


Figure 3.17: Grid fitting stage.

3.7.7 Symbol reconstruction

Following successful grid fitting, the recovered geometric structure is converted into a clean, synthetic representation of the Data Matrix Code suitable for decoding. The two L borders are forced to full occupancy, and the remaining two borders are forced to an alternating timing pattern consistent with the identified L corner. The grid is then padded with a quiet zone and rendered to a binary image

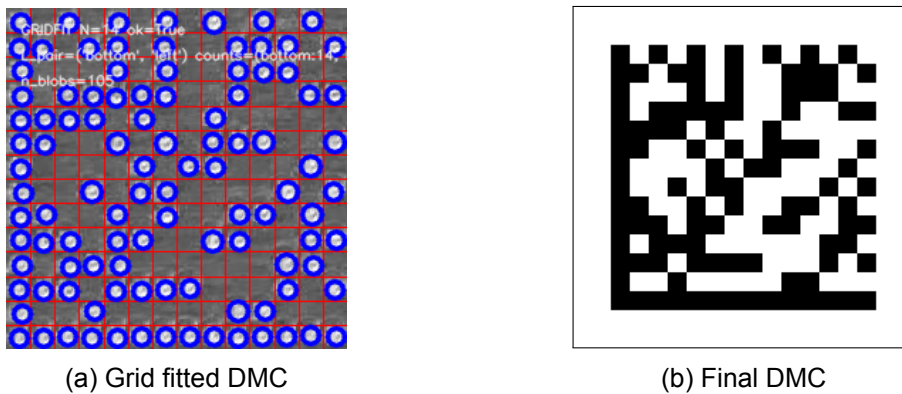


Figure 3.18: Dot peen marked DMC reconstructed to synthetic code.

3.8 Decoding

Following grid fitting and synthetic symbol reconstruction, the resulting Data Matrix Code is passed to a standard ECC200-compliant decoder. In this work, the open-

source `libdmtx` library [20] is used for decoding.

The decoder operates on the reconstructed symbol without additional image-level heuristics or post-processing. By providing a clean and geometrically consistent input, the proposed pipeline reduces many of the failure modes commonly observed when decoding directly from raw or weakly processed industrial images.

Despite the use of error correction, decoding may still fail in cases where the reconstructed symbol exceeds the ECC200 error correction capacity or violates structural assumptions required by the decoder. In such cases, the symbol is explicitly rejected rather than producing an unreliable decoded output.

3.9 Implementation Details

The proposed decoding pipeline is implemented in Python and is structured as a set of modular processing stages. This design allows individual components to be developed, tested, and evaluated independently, while enabling seamless integration into a complete end-to-end system.

Image preprocessing, geometric processing, and rectification are implemented using OpenCV and NumPy. Blob detection relies on the Laplacian-of-Gaussian operator provided by `scikit-image`, while numerical operations such as Principal Component Analysis are performed using standard scientific computing libraries. Learning-based detection and segmentation are implemented using a YOLO-based model trained with the PyTorch framework. Decoding of reconstructed symbols is performed using the `libdmtx` library, which provides a standard-compliant ECC200 decoder.

During inference, the dominant computational cost arises from neural network inference in the detection stage. The subsequent geometric processing steps, including rectification, grid fitting, and reconstruction, are lightweight and incur negligible overhead in comparison. As a result, the pipeline is suitable for practical deployment in offline inspection workflows and can operate on standard CPU hardware during inference.

The current implementation assumes the presence of a single Data Matrix Code per image and targets symbols of size 14×14 and 16×16 . Training of the detection model requires GPU acceleration, while inference and geometric processing can be

performed without specialized hardware. These constraints are consistent with the intended industrial inspection setting and do not limit the generality of the proposed approach.

3.10 Summary

This chapter presented a complete decoding pipeline for industrial Data Matrix Codes captured under challenging imaging conditions. The pipeline combines learning-based detection and segmentation with geometry-driven reasoning to recover the symbol structure prior to decoding.

By separating localization, geometric rectification, and structural reconstruction from the decoding stage, the proposed approach achieves robustness to illumination variability, surface artifacts, and incomplete dot detections while remaining fully compatible with standard ECC200 decoders. The use of explicit geometric constraints enables reliable reconstruction even when local visual information is unreliable.

A visualization of the an image being successfully processed by the pipeline can be found in the E.6 of the Appendix. The effectiveness of the proposed pipeline is evaluated in the following chapter through quantitative experiments on real datasets.

4 Evaluation Setup and Metrics

The proposed decoding pipeline is evaluated exclusively on real industrial images. A key challenge in evaluating industrial Data Matrix decoding is the lack of ground-truth decoded strings for all images. In many cases, the encoded content is not human-readable or cannot be reliably inferred from the image alone. As a result, evaluation is performed in a stage-wise manner, with decoding accuracy assessed only on a controlled subset of real images for which verification is possible.

4.1 Evaluation protocol

Detection and geometric-stage success rates are computed on the full real-image test set of 46 images. Decoding accuracy is evaluated on a subset for which the decoded strings can be verified. Within this subset, the performance of the proposed method is assessed using a sequence of increasingly strict success criteria corresponding to the main stages of the pipeline.

First, detection performance is evaluated by measuring the fraction of images in which a Data Matrix Code is successfully detected by the learning-based segmentation model.

Second, geometric rectification performance is evaluated by measuring the fraction of detected symbols that result in a valid rectified crop after border fitting and perspective normalization.

Third, grid fitting and reconstruction performance are evaluated by measuring the fraction of rectified crops that successfully produce a valid synthetic Data Matrix symbol.

Finally, decoding performance is evaluated by measuring the fraction of reconstructed symbols that are correctly decoded by the ECC200 decoder. Images that fail at earlier stages are excluded from this final evaluation.

4.1.1 Metrics

For each pipeline stage, a success rate is reported as the fraction of images that successfully pass the corresponding stage. This produces a clear breakdown of where failures occur and allows individual components of the pipeline to be analyzed independently.

In addition, results are stratified by image quality based on the code-to-image area ratio introduced in Section 3.2. Images are classified as high-quality or low-quality according to this criterion, allowing pipeline robustness to be evaluated as a function of symbol scale.

4.2 Detection Performance

This section reports the detection performance of the selected learning-based segmentation model on real industrial images. Based on preliminary experiments, the YOLOv11-nano segmentation model trained on real data with augmentation was selected for evaluation. The training details can be found in Appendix's B.2.

Detection performance is evaluated on a held-out real-image test set consisting of 46 images. Successful detection is defined as the presence of a predicted segmentation mask that sufficiently overlaps the Data Matrix Code region to enable correct cropping and rectification in later pipeline stages. Precise boundary alignment is not required at this stage.

Detection correctness is assessed using available annotations and visual inspection, without relying on successful decoding of the encoded content.

4.2.1 Overall detection results

In all 46 test images, at least one Data Matrix Code was successfully detected.

Four images contained false positive detections. In one case, a false positive detection occurred on an image region not containing a Data Matrix Code. In the remaining cases, additional secondary detections were produced alongside a correct detection of the true code, with differing predicted marking classes. These secondary detections were associated with higher confidence scores than the primary detection.

Metric	Value
Total images	46
Detected images	46
Detection success rate	100.0%
Images with false positive detections	4
Images with false classification	3
Images with non DMC detection	1

Table 4.1: Overall detection performance on the real-image test set.

4.2.2 Detection performance by marking type

The test set contains both laser-marked and dot-peened Data Matrix Codes. Detection results for each marking type are summarized in the table below.

Marking type	Total images	Detected	Success rate
Laser-marked	23	23	100.0%
Dot-peened	23	23	100.0%

Table 4.2: Detection performance by marking type.

4.2.3 Detection performance by image quality

To further characterize detection robustness, results are stratified by image quality using a threshold of 5% on the code-to-image area ratio. As explained in Section 3.2.4, images with a ratio below this threshold are classified as low quality, while images above the threshold are classified as high quality.

Quality class	Total images	Detected	Success rate
Low quality ($\leq 5\%$)	30	30	100.0%
High quality ($> 5\%$)	16	16	100.0%

Table 4.3: Detection performance by image quality.

4.3 Geometric rectification performance

Geometric rectification converts the predicted segmentation mask into a canonical, square crop of the symbol by estimating a quadrilateral boundary and applying a projective warp. Rectification is considered successful if the resulting crop contains the full Data Matrix Code, including the finder pattern, such that it can be forwarded

to the grid fitting stage. Crops that truncate the symbol or contain insufficient symbol content are considered failures at this stage.

Starting with the 46 test images, where detection was available, after rectification, all 46 detections produced a rectified crop. Two of these crops were excluded from further processing: one corresponding to a false positive detection and one where the rectified crop contained insufficient symbol content. The remaining 44 crops were considered valid rectified symbols and were forwarded to grid fitting.

Metric	Value
Images with detection available	46
Detections producing a rectified crop	45
Excluded rectified crops (insufficient content)	1
Valid rectified crops	44
Rectification success rate	95.7%

Table 4.4: Geometric rectification performance on the real-image test set. A valid rectified crop must contain the full symbol region and be suitable for grid fitting.

4.4 Grid fitting and synthetic reconstruction performance

The grid fitting stage estimates the internal module layout of each rectified symbol and determines whether a synthetic Data Matrix representation can be constructed. Grid fitting is considered successful if a consistent grid structure is identified and validated, allowing the symbol to be reconstructed into a synthetic image suitable for decoding.

Following geometric rectification, 23 of the valid rectified crops corresponded to dot-peened symbols. Of these, 16 were successfully reconstructed into synthetic Data Matrix images using the dot-peen processing branch. The remaining 7 dot-peened symbols did not satisfy the grid fitting criteria for the dot-peen branch and were instead forwarded to the laser-marked preprocessing branch.

All rectified laser-marked symbols were processed using the laser branch of the pipeline, as expected.

Pipeline stage	Successful images	Success rate
Detection	46 / 46	100.0%
Valid rectified crop	44 / 46	95.7%
Synthetic DMC generation	16 / 23	69.6%
Preprocessed laser DMC	23 / 23	100.0%

Table 4.5: Stage-wise success rates of the proposed pipeline on real industrial images.

The quality distribution of the dot-peened symbols is summarized as follows. Among the 16 dot-peened symbols successfully reconstructed into synthetic images, 10 were classified as low quality and 6 as high quality. Among the 7 dot-peened symbols that failed dot-peen grid fitting, 5 were low quality and 2 were high quality.

Outcome	Total	Low quality	High quality
Reconstructed as dot-peen	16	10	6
Failed dot-peen grid fitting	7	5	2
Total dot-peened symbols	23	15	8

Table 4.6: Grid fitting outcomes for dot-peened symbols after geometric rectification.

4.5 Performance on the validated decoding dataset

4.5.1 Validated decoding dataset

In addition to the full test set evaluation, a separate subset of real industrial images is used to evaluate decoding performance under verifiable conditions. This validated dataset is constructed by identifying and selecting images in which the entire Data Matrix Code is clearly visible, does not violate ECC200 structural requirements, and for which the encoded string can be reliably determined from the image. It is important to mention that these images have not been seen by the model at any point. Therefore, this separation supports transparent evaluation of the full pipeline, since restricting the validated set reduces the risk that prior exposure at earlier stages could bias downstream success rates and affect the interpretation of end-to-end decoding performance. The resulting subset consists of 36 images, evenly split between dot-peened and laser-marked Data Matrix Codes.

4.5.2 Pipeline performance

Starting from the validated dataset, 35 out of 36 images were successfully detected, corresponding to a detection success rate of 97.2%. One image failed at the detection stage and was excluded from further processing.

All 35 detected symbols were successfully rectified, producing valid rectified crops containing the full Data Matrix Code region.

Among the rectified symbols, 18 corresponded to dot-peened markings and 17 to laser-marked markings. Of the dot-peened symbols, 14 were successfully reconstructed into synthetic images using the dot-peen processing branch, while the remaining 4 were forwarded to the laser-marked processing branch. All laser-marked symbols were processed following the laser branch of the pipeline.

Decoding performance is evaluated on the validated dataset using an ECC200 compliant decoder. A decoding attempt is considered successful if the decoded string matches the known ground-truth content of the symbol.

Out of the 36 validated images, 19 were successfully decoded, corresponding to an overall decoding success rate of 52.8%. Of these, 10 corresponded to dot-peened symbols and 9 to laser-marked symbols.

Pipeline stage	Images	Stage success	Cumulative success
Validated dataset	36	100.0%	100.0%
Detected	35	97.2%	97.2%
Valid rectified crops	35	100.0%	97.2%
Grid fitting	31	88.6%	86.1%
Successfully decoded	19	61.3%	52.8%

Table 4.7: Stage-wise progression of images through the pipeline on the validated decoding dataset. Stage success rate is computed relative to the previous pipeline stage, while cumulative success rate is computed relative to the validated dataset.

Marking type	Images	Decoded	Success rate
Dot-peened	18	10	55.6%
Laser-marked	18	9	50.0%
Total	36	19	52.8%

Table 4.8: Decoding performance by marking type on the validated dataset.

Category	Images	Decoded	Success rate
Dot-peened (low quality)	11	6	54.5%
Dot-peened (high quality)	7	4	57.1%
Laser-marked (low quality)	10	5	50.0%
Laser-marked (high quality)	7	4	57.1%

Table 4.9: Decoding performance stratified by image quality on the validated dataset.

The computational performance of the proposed pipeline is evaluated on the validated dataset. Reported timings correspond to the average processing time per image, measured over all successfully processed images.

Pipeline stage	Average time (s)
Preprocessing	0.341
Detection	5.051
Rectification	1.203
Grid fitting	2.518
Decoding	1.324
Total pipeline time	10.513

Table 4.10: Average runtime per pipeline stage on the validated dataset. The run was executed on a CPU.

5 Discussion

5.1 Analysis of Failure Cases

This section examines representative failure cases in the proposed decoding pipeline, highlighting how different sources of degradation impact individual processing stages under industrial imaging conditions. Some visual examples of such failures for both dot-peened and laser-marked Data Matrix Codes can be found in Section E of the Appendix.

5.1.1 Detection-related failures

Detection failures in industrial imagery arise primarily from a combination of data scarcity, challenging acquisition conditions, and extreme symbol scale variations. While the detection stage achieved a high recall on the evaluated datasets (Section 4.2), the observed failure cases highlight structural limitations of the problem setting rather than weaknesses of the detection model itself.

A key contributing factor is the limited availability of annotated training data. Deep learning-based detection models typically benefit from large and diverse datasets in order to learn robust and invariant feature representations. In this work, only a small number of real industrial images were available for training, restricting the model's exposure to rare symbol appearances, extreme lighting conditions, and uncommon surface artifacts. This constraint is particularly relevant for industrial inspection tasks, where data collection and annotation are costly and time-consuming.

Image quality further impacts detection reliability. As shown in Section 3.2.4 and quantified in Tables 4.3 and 4.9, the majority of images in the dataset fall into the low-quality category based on the code-to-image area ratio. In such cases, the Data Matrix Code occupies only a small fraction of the image, resulting in limited spatial resolution for the symbol and reduced contrast between code elements and the background. These conditions amplify the effect of noise, glare, and surface texture, making reliable localization more difficult.

In addition to scale and quality, industrial surface characteristics introduce visual ambiguity that is uncommon in printed barcode datasets. Dot-peened and laser-marked symbols are often surrounded by scratches, machining marks, engravings, or highly reflective regions that can produce structures visually similar to Data Matrix patterns. This occasionally leads to false positive detections or misclassification of the marking type, as reported in Section 4.2. While such errors do not necessarily prevent subsequent processing, they illustrate the inherent difficulty of robust detection in unconstrained industrial environments.

Overall, detection-related failures are rare in the evaluated datasets, but when they occur, they are predominantly associated with extreme imaging conditions, limited training data diversity, and very small or low-contrast symbols.

5.1.2 Geometric rectification failures

Geometric rectification failures occur when the segmentation mask produced by the detection model does not support a stable estimation of a quadrilateral enclosing the Data Matrix Code. Since rectification operates exclusively on the predicted mask geometry, failures at this stage are driven by limitations in mask structure rather than image appearance.

The dominant source of rectification failure arises from ambiguity in the geometric structure of the predicted mask. When the mask exhibits highly irregular shapes, cavities, or fragmented regions, these operations may fail to produce a stable quadrilateral. In such cases, polygon simplification can introduce spurious vertices or suppress true corner points, while corner extraction and ordering may yield incorrect or ambiguous corner configurations. This can result in false corner assignments, incorrect corner ordering, or truncated perspective warps of the symbol region.

Rectification may also fail when the mask boundary lacks distinct geometric cues corresponding to the true symbol corners. Smooth or rounded outlines, elongated attachments, or locally noisy boundaries can cause false corner selection or ambiguous corner ordering, resulting in truncated or misaligned warps. An example can be found in the Appendix E.3.

Image quality plays a secondary and indirect role in rectification robustness. Unlike detection or decoding, rectification is largely insensitive to local contrast, noise, or surface texture, as it relies solely on the spatial extent and geometry of the segmentation

mask. However, when the symbol occupies only a small fraction of the image, minor localization errors in the mask boundary can be amplified during polygon simplification and homography estimation. In these cases, small geometric inaccuracies can lead to disproportionate cropping errors after normalization, increasing the likelihood of rectification failure.

Overall, rectification failures are relatively infrequent compared to later pipeline stages, but they predominantly occur when the predicted mask geometry does not support a stable and rectangular approximation of the symbol region. The conservative rejection criteria applied at this stage ensure that only geometrically reliable crops are forwarded to grid fitting.

5.1.3 Grid fitting failures

Grid fitting failures occur when a rectified symbol cannot be mapped to a consistent internal grid structure compatible with the Data Matrix specification. At this stage, processing relies on polarity normalization, blob detection, border fitting, and structural validation, as described in Section 3.7.

A common source of failure is unstable blob detection caused by residual polarity inconsistencies or interference from surface artifacts. Although a dedicated polarity normalization step is applied prior to blob detection, this procedure is intentionally simple to avoid overfitting to specific lighting conditions. When blob responses are dominated by glare, scratches, or noise rather than true dot-peen marks, subsequent border estimation becomes unreliable. This may result in incorrect line fitting or insufficient support for valid border hypotheses.

Failures also occur when the physical structure of the symbol does not support reliable blob extraction. In dot-peened markings, individual dots may be missing, merged, or unevenly illuminated, resulting in incomplete or fragmented blob candidates. When the density or spatial distribution of detected blobs is insufficient, border fitting becomes unreliable, as the estimated reference lines cannot be anchored to a stable set of observations. This is particularly problematic when portions of the L-shaped finder pattern are degraded or absent.

Image quality plays a significant role at this stage. For symbols occupying a small fraction of the image, the number of pixels per module is limited, reducing the robustness of blob localization and increasing sensitivity to local defects. Under these

conditions, small errors in blob position or border estimation can propagate through the grid fitting process, leading to unstable or invalid grid configurations.

Finally, grid fitting failures may result from violations of structural assumptions inherent to the Data Matrix symbology. Overlapping dots, missing quiet zones, or distorted module arrangements can prevent the enforcement of grid regularity constraints, even when earlier processing stages succeed. Such cases exceed the tolerance of the grid fitting procedure and are therefore rejected.

Overall, grid fitting represents the stage at which intensity-based feature extraction and structural validation intersect. Failures at this point reflect the combined sensitivity of polarity handling, blob detection, and border fitting to physical degradation and imaging artifacts commonly encountered in industrial markings.

5.1.4 Decoding failures

Decoding failures occur when a reconstructed Data Matrix symbol cannot be decoded by the ECC200-compliant decoder, despite successful detection and geometric rectification. At this stage, failures primarily reflect limitations in the accuracy of the reconstructed binary grid and the tolerance of the decoding process, rather than geometric ambiguity.

A common cause of decoding failure is residual reconstruction error in the module grid. Illumination effects, imperfect polarity normalization, and physical marking artifacts can lead to missed or spurious blob detections, which propagate to incorrect module assignments. When the resulting codeword errors exceed the error correction capacity of the ECC200 specification, decoding fails even if the grid appears geometrically consistent.

Failures may also arise from mismatches in the reconstructed symbol representation. In particular, if grid fitting converges to an incorrect interpretation of the marking type, the reconstructed grid may violate the contrast or sampling assumptions required by the decoder. In such cases, decoding fails despite successful rectification and grid estimation.

Decoding robustness is further constrained by the decoder implementation. In this work, decoding is performed using the open-source `libdmtx` library [20], which implements ECC200 decoding without additional heuristics or learned priors. While

standards-compliant and widely used, prior comparisons indicate reduced robustness for small symbols, low-contrast markings, and direct part marking scenarios compared to proprietary decoders [21, 22]. A comparison of open-source decoders is provided in Appendix D.1.

For laser-marked symbols, insufficient module contrast and local intensity ambiguity are frequent contributors to failure. Shallow engraving, glare, and surface texture can weaken module separability, leading to incorrect binary sampling and decoder rejection.

Several failure cases in the validated dataset involve symbols that appear visually clean after reconstruction but remain undecodable. These cases indicate that decoding failure is not necessarily the result of earlier pipeline errors, but rather reflects the strict structural and error correction constraints of ECC200 decoding under industrial marking conditions [1, 20].

Overall, decoding represents the most restrictive stage of the pipeline, where the reconstructed grid must satisfy both structural validity and decoder tolerance requirements. Failures at this stage motivate the conservative handling of undecoded symbols adopted in this work.

5.2 Design Choices and Trade-offs

This section discusses the design choices and trade-offs of the proposed decoding pipeline in relation to the research questions defined in Section 1.5. In particular, the results presented in Chapter 4 demonstrate that learning-based segmentation can reliably localize Data Matrix Codes under challenging industrial conditions (RQ1.1), while explicit geometric rectification and grid reconstruction significantly improve robustness for dot-peen markings (RQ1.2). The stage-wise evaluation further highlights how failures emerge at different points in the pipeline depending on image quality and physical degradation, directly addressing RQ1.3.

5.2.1 Geometry-first vs learning-first

The proposed pipeline follows a geometry-first design, where explicit geometric and structural constraints are applied before decoding, instead of relying on a fully learning-

based, end-to-end solution. This design choice is motivated by the characteristics of the available data and the requirements of the industrial setting considered in this work.

Learning-based approaches can perform well when large and diverse annotated datasets are available. However, as discussed in Section 3.2, the number of real industrial images available for training is limited, and the visual appearance of the symbols varies significantly due to surface texture, marking process, and illumination. Under these conditions, a fully learning-based decoding approach would be difficult to train reliably and would likely generalize poorly to unseen cases.

In contrast, a geometry-first approach allows known properties of Data Matrix Codes to be enforced explicitly through geometric rectification and grid fitting, as described in Chapter 3. This makes it possible to reject invalid symbols early and to isolate failures to specific stages of the pipeline. As shown by the stage-wise results in Section 4.5, this structured processing enables robust handling of challenging inputs while preventing error propagation to later stages.

The main trade-off of this design choice is reduced flexibility. Strict geometric constraints may reject symbols that could potentially be decoded by a highly specialized learning-based model. However, given the limited training data and the need for predictable and interpretable behavior, robustness and controlled failure handling were prioritized over maximizing recall.

Overall, the geometry-first design reflects a practical balance between robustness, interpretability, and feasibility under the constraints of the available data and the industrial imaging conditions addressed in this work.

5.2.2 Segmentation vs bounding boxes

As described in Section 3.5, segmentation-based detection was used instead of standard bounding box localization to identify Data Matrix Code regions. This design choice was motivated by the downstream requirement for accurate geometric rectification, where imprecise localization can directly affect grid fitting and decoding stability.

From a practical perspective, segmentation provides a dense, pixel-level representation of the symbol region, enabling a tighter and more accurate estimation of its

spatial extent. This reduces the inclusion of irrelevant background regions in the rectified crop and improves robustness when symbols are rotated, partially occluded, or surrounded by surface structures. In the proposed pipeline, this directly contributed to more stable corner extraction and reduced the likelihood of truncation or misalignment during perspective warping.

In contrast, bounding box-based localization was observed to be less reliable in the industrial setting considered in this work. Loose or misaligned bounding boxes frequently included surrounding scratches, glare, or machining marks, increasing ambiguity during geometric rectification and grid fitting. Such errors were difficult to correct in later stages and often propagated through the pipeline, leading to unstable reconstruction or rejection.

The primary trade-off of segmentation-based detection is increased model complexity and computational cost compared to bounding box detection. However, given that detection is performed only once per image and that subsequent geometric processing dominates decoding reliability, this trade-off was considered acceptable. The experimental results in Chapter 4 demonstrate that the improved spatial precision provided by segmentation outweighs its additional complexity in the context of the proposed geometry-first decoding pipeline.

5.2.3 Border fitting before grid fitting

In the proposed pipeline, border fitting is performed before attempting full grid fitting. This design choice was made to reduce ambiguity and instability when estimating the internal structure of the Data Matrix Code, particularly in challenging industrial conditions.

Grid fitting relies on the presence of a consistent reference frame to define module orientation, scale, and alignment. Without prior border estimation, grid fitting must consider a large number of possible configurations, many of which are incompatible with the actual symbol structure. Early experiments that attempted to fit the grid directly from blob detections resulted in unstable or inconsistent grid hypotheses, especially when border dots were partially missing or corrupted.

By fitting the border first, the pipeline establishes a geometric reference for the symbol through the L-shaped finder pattern. This constrains the subsequent grid fitting step by fixing the grid orientation and limiting the search space for valid grid configu-

rations. As a result, grid fitting becomes more stable and less sensitive to local noise or spurious blob detections.

This staged approach is particularly important for dot-peened symbols, where border dots may be uneven, incomplete, or affected by surface artifacts. In these cases, attempting grid fitting without a reliable border reference often leads to incorrect grid alignment or failure. By explicitly validating the border before proceeding, the pipeline can either recover a stable grid or reject the symbol early, preventing error propagation to later stages.

The main trade-off of this design choice is increased strictness. Symbols with weak or partially degraded borders, pictures with glare and bad quality rectification, may be rejected even if parts of the internal grid are visible. However, given the downstream reliance on a consistent grid structure for decoding, this conservative strategy improves overall robustness and predictability of the pipeline.

5.2.4 Polarity normalization strategy

The proposed pipeline includes a polarity normalization step prior to blob detection and grid fitting, as described in Chapter 3.7. This step aims to ensure a consistent foreground–background relationship, where symbol modules appear brighter than the surrounding surface, which is required by the subsequent intensity-based blob detection stage.

A deliberately simple polarity normalization strategy was adopted. Rather than applying a global inversion or learning-based polarity estimation, the approach relies on local intensity statistics to determine whether inversion is required. This restriction is guided by typical glare behavior observed in industrial imagery, and it aims to avoid overfitting to specific lighting conditions and to maintain predictable behavior across diverse industrial surfaces.

While this strategy improves robustness in many cases, it does not guarantee correct polarity normalization under all conditions. In images with strong glare, uneven illumination, or mixed surface reflectance, local intensity cues may be ambiguous, resulting in residual polarity inconsistencies. As discussed in Section 5.1.3, incorrect polarity can cause the blob detector to respond to background structures or specular highlights instead of true symbol modules, leading to unstable border fitting or invalid grid hypotheses.

The primary trade-off of this design choice is simplicity versus completeness. More sophisticated polarity estimation techniques could potentially improve performance in challenging cases but would introduce additional complexity and risk over-specialization. Given the limited training data and the need for robust, interpretable behavior, the chosen polarity normalization strategy represents a practical compromise between reliability and generality.

5.2.5 Conservative validation and rejection strategy

Throughout the proposed pipeline, conservative validation and rejection criteria are applied at multiple stages, including rectification, grid fitting, and decoding. Rather than attempting to recover a result from uncertain or unstable intermediate representations, the pipeline explicitly rejects symbols that do not satisfy basic geometric or structural consistency checks.

This strategy is motivated by the cumulative nature of errors in multi-stage processing pipelines. Allowing unreliable intermediate results to propagate can lead to unstable behavior in later stages, where failures are harder to detect and interpret. By contrast, early rejection localizes failure modes and prevents downstream stages from operating on invalid inputs, as discussed in Section 5.1.2 and Section 5.1.3.

Conservative validation is particularly important for industrial Data Matrix Codes, where physical degradation, surface artifacts, and incomplete markings are common. In such cases, aggressive recovery attempts may produce misleading or incorrect decoded outputs. The adopted rejection strategy favors reliability and interpretability over maximizing the number of decoded symbols.

The main trade-off of this approach is reduced recall. Some symbols that contain partial or ambiguous information may be rejected even though a decoder could potentially recover a valid payload. However, as reflected in the decoding results presented in Chapter 4, prioritizing controlled failure behavior leads to more predictable system performance and avoids unreliable outputs.

Overall, the conservative validation and rejection strategy supports the design goal of robust and transparent industrial decoding, ensuring that only symbols meeting minimum geometric and structural requirements are forwarded for decoding.

5.3 Limitations

While the proposed pipeline demonstrates robust performance on challenging industrial Data Matrix imagery, several limitations should be acknowledged.

First, the pipeline assumes the processing of a single Data Matrix Code per image. Multiple codes contained in a single image, are not explicitly handled. Although the detection stage could be extended to support multiple instances, the downstream geometric and decoding stages are currently designed for single-symbol processing.

Second, the pipeline operates under a fixed set of supported grid sizes. While this covers the most common industrial cases encountered in the dataset, symbols with uncommon or unexpected dimensions may not be decoded successfully. Grid size estimation is therefore limited by the predefined configurations used during validation.

Third, the approach is sensitive to severe physical degradation. Symbols exhibiting extensive mechanical damage, heavy surface contamination, or extreme wear may violate structural assumptions required for grid fitting and decoding. In such cases, conservative rejection is applied to avoid unreliable outputs, which reduces recall under extreme conditions.

Fourth, decoding is performed without semantic validation of the recovered payload. The pipeline verifies structural and geometric consistency but does not check whether the decoded string conforms to application-specific formats or constraints. As a result, semantic correctness is assumed once decoding succeeds.

Finally, although runtime performance is acceptable for offline processing, the pipeline is not optimized for real-time deployment. The multi-stage design introduces computational overhead, particularly during detection and grid fitting, which may limit applicability in time-critical scenarios without further optimization.

5.4 Future Work

The proposed pipeline provides a robust and interpretable solution for decoding industrial Data Matrix Codes under challenging conditions. Several extensions can be explored to further improve its flexibility and performance while preserving its core design principles.

One natural extension is adaptive grid size estimation. The current pipeline operates on a fixed set of supported grid sizes, which covers common industrial cases but limits generality. Extending the grid fitting stage to evaluate a broader range of grid hypotheses, potentially guided by structural confidence measures, could improve decoding performance for less common symbol formats.

Improvements to polarity normalization also represent a promising direction. While the current strategy favors simplicity and robustness, more adaptive methods could be explored to better handle mixed illumination and complex glare patterns. Such approaches could leverage multi-scale intensity analysis or learned polarity classification while remaining decoupled from the core geometric processing stages.

Grid fitting could further benefit from confidence-weighted blob support. Rather than treating all detected blobs equally, incorporating measures of blob reliability based on intensity, shape, or spatial consistency could improve border estimation and grid stability, particularly for heavily degraded dot-peened symbols.

The pipeline could also be extended to support multiple Data Matrix Codes per image. While the detection stage is inherently capable of handling multiple instances, downstream geometric and decoding stages would require instance-wise processing and result aggregation to support multi-code scenes.

Future work could benefit from the development of a more systematically curated dataset for industrial Data Matrix Codes. A dataset designed with controlled variation in marking quality, symbol size, and imaging conditions would support more rigorous evaluation and facilitate improved generalization analysis of decoding pipelines.

Finally, limited feedback from the decoding stage could be used to refine earlier processing steps. Decoder success or failure provides information about grid validity that could be exploited to guide alternative grid hypotheses or trigger localized reprocessing, without transitioning to a fully end-to-end learning framework.

These extensions represent incremental improvements that build on the existing geometry-first design and could further enhance robustness and applicability in industrial deployment scenarios.

6 Conclusion

This thesis presented a robust, geometry-first pipeline for decoding industrial Data Matrix Codes from real-world imagery. The proposed approach addresses the challenges of direct part marking, including low contrast, surface reflections, mechanical degradation, and limited availability of annotated training data.

A learning-based segmentation model was used to localize symbols, while subsequent stages relied on explicit geometric rectification, border fitting, grid reconstruction, and ECC200 decoding. By combining learning-based detection with rule-based geometric processing, the pipeline maintains interpretability and allows failures to be localized and handled explicitly at each stage.

Experimental evaluation on real industrial images demonstrated that the proposed pipeline achieves reliable detection and decoding performance across both laser-marked and dot-peened symbols. Stage-wise analysis highlighted grid fitting and decoding as the primary sources of failure, reflecting the inherent difficulty of recovering structured symbolic information from physically degraded markings. The conservative validation strategy adopted throughout the pipeline ensured predictable behavior and avoided unreliable outputs.

The results show that explicit geometric reasoning remains a viable and effective approach for industrial Data Matrix decoding, particularly in scenarios where data is limited and symbol structure is known. Rather than relying on end-to-end learning, the proposed pipeline leverages domain knowledge to achieve robustness and transparency under challenging conditions.

Overall, this work demonstrates that combining learning-based localization with geometry-driven reconstruction provides a practical and interpretable solution for industrial Data Matrix decoding, and establishes a solid foundation for future extensions in industrial inspection and identification systems.

Bibliography

- [1] *Information technology — Automatic identification and data capture techniques — Data Matrix bar code symbology specification*. ISO/IEC, 2024.
- [2] KEYENCE Corporation. *Basic Practice of 2D Codes VOL. 1: Types of 2D Codes*. 2020. URL: https://www.keyence.eu/ss/products/auto_id/codereader/basic_2d/.
- [3] Alan Richter. “Enter the Matrix”. In: *Cutting Tool Engineering* (2011).
- [4] M. Grasso and B. M. Colosimo. “Quality assessment of direct part marking codes”. In: *Journal of Manufacturing Systems* (2017).
- [5] Néstor Falcón et al. “Image Processing Techniques for Braille Writing Recognition”. In: *Lecture Notes in Computer Science (EUROCAST)*. 2005.
- [6] Kirill Smelyakov et al. “Braille Character Recognition Based on Neural Networks”. In: *IEEE International Conference on Data Stream Mining and Processing*. 2018.
- [7] Zezheng Meng et al. “Braille Character Segmentation Algorithm Based on Gaussian Diffusion”. In: *Computers, Materials & Continua* (2024).
- [8] Licheng Liao, Jianmei Li, and Changhou Lu. “Data Extraction Method for Industrial Data Matrix Codes Based on Local Adjacent Modules Structure”. In: *Applied Sciences* (2022).
- [9] Aidan Stocks. “Data Matrix Code Decoding in Industrial Environments”. MA thesis. 2025.
- [10] Keiron O’Shea and Ryan Nash. “An Introduction to Convolutional Neural Networks”. In: *arXiv preprint arXiv:1511.08458* (2015). URL: <https://arxiv.org/abs/1511.08458>.
- [11] Yann LeCun et al. “Gradient-Based Learning Applied to Document Recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324. DOI: 10.1109/5.726791.
- [12] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. 3rd ed. Pearson, 2008.
- [13] OpenCV Developers. *OpenCV Documentation*. <https://docs.opencv.org/>. 2024.
- [14] Jean Serra. *Image Analysis and Mathematical Morphology*. Academic Press, 1982.

- [15] Mark de Berg et al. *Computational Geometry: Algorithms and Applications*. Springer, 2008.
- [16] David H Douglas and Thomas K Peucker. “Algorithms for the reduction of the number of points required to represent a digitized line or its caricature”. In: *Cartographica* 10.2 (1973), pp. 112–122.
- [17] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. 2nd ed. Cambridge University Press, 2003.
- [18] Tony Lindeberg. “Feature detection with automatic scale selection”. In: *International Journal of Computer Vision* (1998).
- [19] Kenichi Kanatani. *Statistical Optimization for Geometric Computation*. Elsevier, 1996.
- [20] Mike Schepers and contributors. *libdmtx: Data Matrix Encoding and Decoding Library*. 2023. URL: <https://github.com/dmtx/libdmtx>.
- [21] Dynamsoft. *Data Matrix Reading Benchmark and Comparison*. 2022. URL: <https://www.dynamsoft.com/codepool/data-matrix-reading-benchmark-and-comparison.html>.
- [22] Dynamsoft. *Choosing the right alternative to libdmtx for reading Data Matrix codes*. Accessed: 2026-01-23. 2025. URL: <https://www.dynamsoft.com/blog/insights/libdmtx-vs-dynamsoft-data-matrix-scanner/>.
- [23] Cartography Playground. *Douglas–Peucker Algorithm Interactive Playground*. Accessed: 2026-01-31. 2023. URL: <https://cartography-playground.gitlab.io/playgrounds/douglas-peucker-algorithm/>.
- [24] ZBar Project. *ZBar Bar Code Reader*. 2022. URL: <https://github.com/mchehab/zbar>.
- [25] ZXing Project. *ZXing-C++: Multi-format 1D/2D Barcode Image Processing Library*. 2023. URL: <https://github.com/zxing-cpp/zxing-cpp>.
- [26] Mike Schepers and contributors. *libdmtx Documentation and Design Notes*. 2023. URL: <https://libdmtx.sourceforge.net/>.

A Sudo-code of the pipeline scripts

Algorithm 1: Gaussian Correction

Input: Input RGB image I_{RGB}

Output: Corrected grayscale image I_{corr}

Convert I_{RGB} to grayscale I ;

Compute Gaussian blur $G \leftarrow \text{GaussianBlur}(I)$;

if mean intensity of I is high **then**

$I_{\text{corr}} \leftarrow I / (G + \epsilon)$;

else

$I_{\text{corr}} \leftarrow I - G$;

Rescale I_{corr} to $[0, 255]$;

return I_{corr} ;

Algorithm 2: Rectification from Segmentation Mask

Input: Input image I , segmentation polygon P

Output: Rectified square crop I_{warp}

Convert polygon P to pixel coordinates;

Rasterize P into binary mask M ;

Apply morphological opening to remove thin bridges;

Keep the largest connected component of M ;

Apply morphological closing to restore small gaps;

Extract outer contour C from cleaned mask;

Compute convex hull of C ;

Simplify hull using polygon approximation;

Select four sharpest corner points;

Order corners as (top-left, top-right, bottom-right, bottom-left);

Expand quadrilateral by class-dependent padding;

Compute projective transform and warp image to square grid;

return I_{warp} ;

Algorithm 3: Grid Fitting and Synthetic Data Matrix Generation

Input: Rectified image I_{warp} , class label

Output: Synthetic binary Data Matrix image

if *marking type is laser* **then**

 Apply contrast enhancement (CLAHE);

 Export enhanced image;

return

else

 Normalize polarity using quadrant-based histogram analysis;

Detect dot centers using Laplacian-of-Gaussian filtering;

Estimate module pitch from nearest-neighbor distances;

Iteratively fit border lines using expanding blob bands;

Identify adjacent L-shaped border pair;

Offset fitted borders outward to define crop region;

Warp image to canonical grid coordinates;

foreach $N \in \{16, 14\}$ **do**

 Assign detected dots to an $N \times N$ grid;

if *L-pattern consistency is satisfied* **then**

 Select grid size N ;

break;

Enforce L-pattern and timing pattern constraints;

Add quiet zone padding around grid;

Render synthetic binary image;

return synthetic Data Matrix image;

Algorithm 4: End-to-End Data Matrix Decoding Pipeline

Input: Input image path

Output: Decoded Data Matrix string (if successful)

Apply Gaussian preprocessing (Algorithm 1);

Run YOLO segmentation to localize Data Matrix symbol;

if *no detection is found* **then**

return failure;

else

 Rectify detected region (Algorithm 2);

Perform grid fitting and synthetic generation (Algorithm 3);

Decode synthetic image using libdmtx;

if *decoding is successful* **then**

 Save decoded string;

else

return decoded result;

A.1 Code Availability

The complete implementation of the proposed decoding pipeline, including preprocessing, detection, rectification, grid fitting, and evaluation scripts, is available at:

<https://github.com/tottispavlou/Data-Matrix-Decoding-in-machined-components>

The repository includes documentation and instructions for reproducing the experimental results reported in this thesis.

B Detection Model

B.1 YOLO Detection and Segmentation

Figure B.1: High level conceptual illustration of the YOLO object detection pipeline used for explanatory purposes.

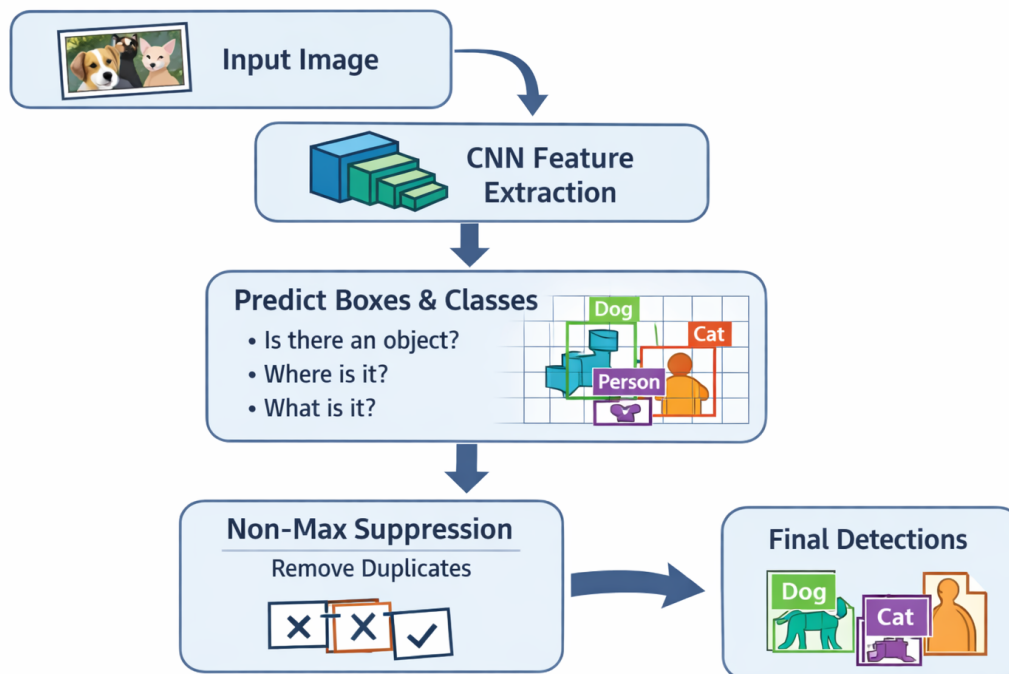
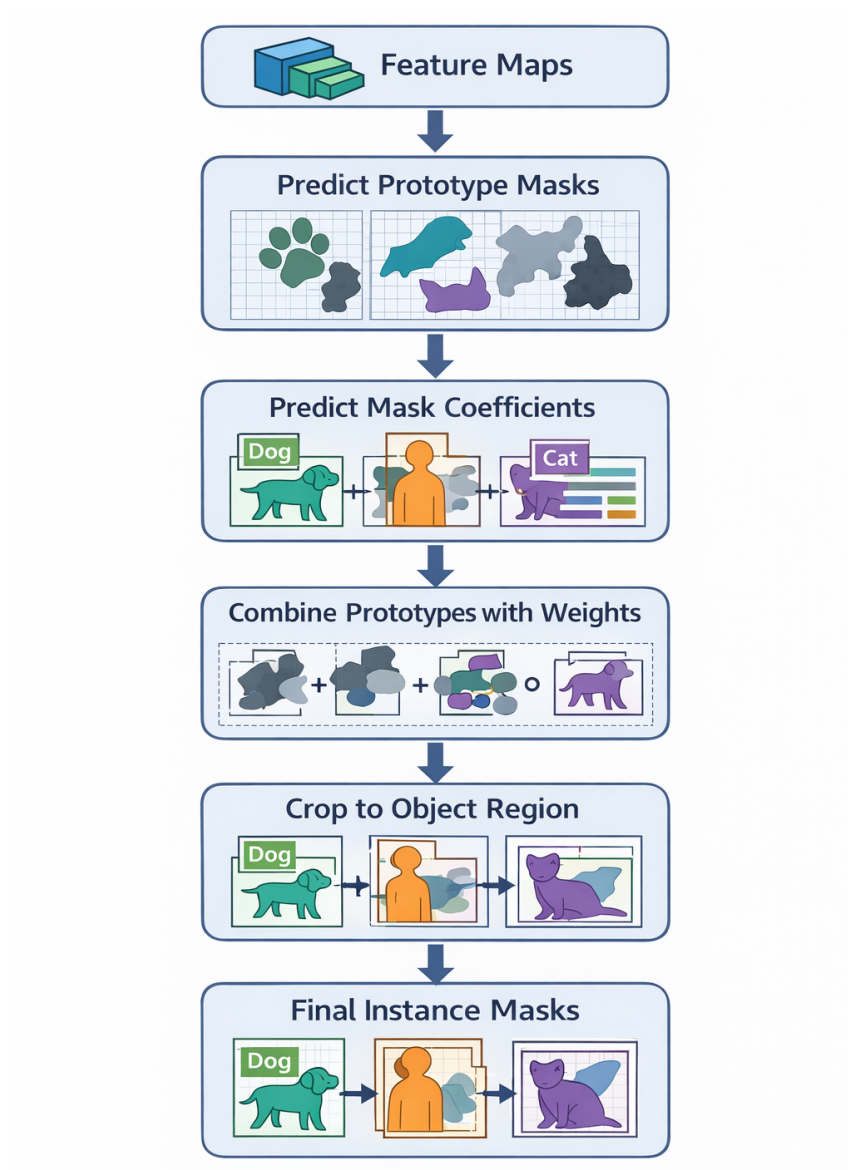


Figure B.2: High level conceptual illustration of the YOLO segmentation pipeline. The Feature Maps are resulting from the CNN Feature Extraction from Figure B.1.



B.2 Effect of synthetic training data

Figure B.3: Detection performance of the YOLO11m segmentation model as a function of synthetic training data. Results are shown for models trained with and without built-in augmentation.

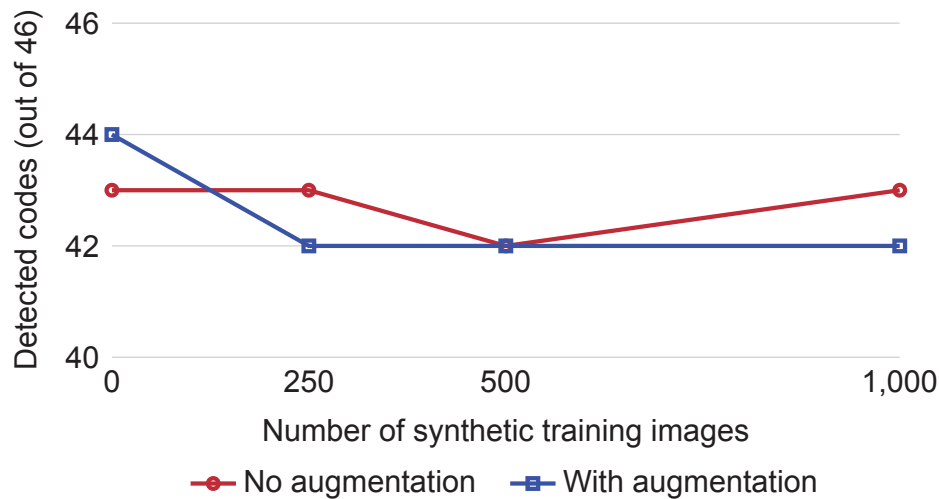


Figure B.4: Recall as a function of synthetic data volume for YOLO11m segmentation models. Recall saturates beyond a small number of synthetic samples and does not improve with additional augmentation.

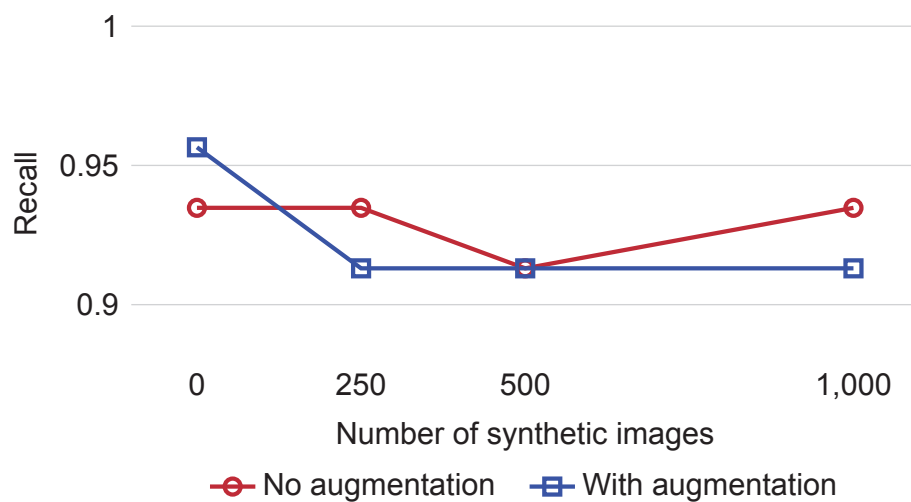
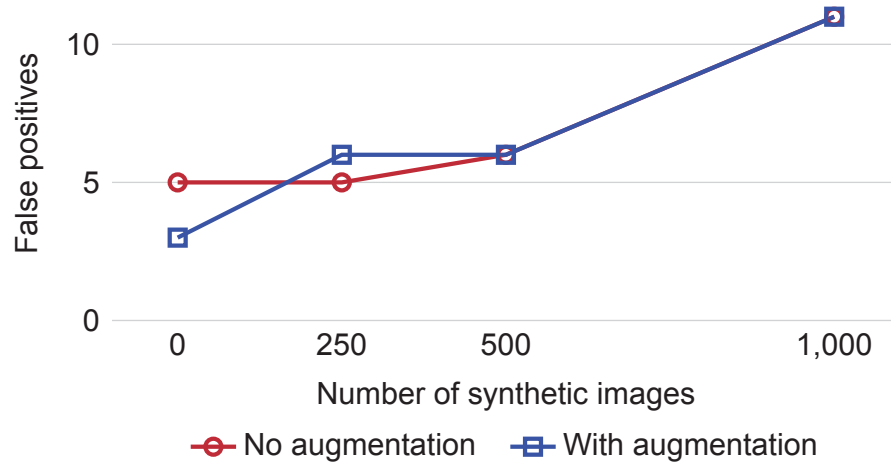
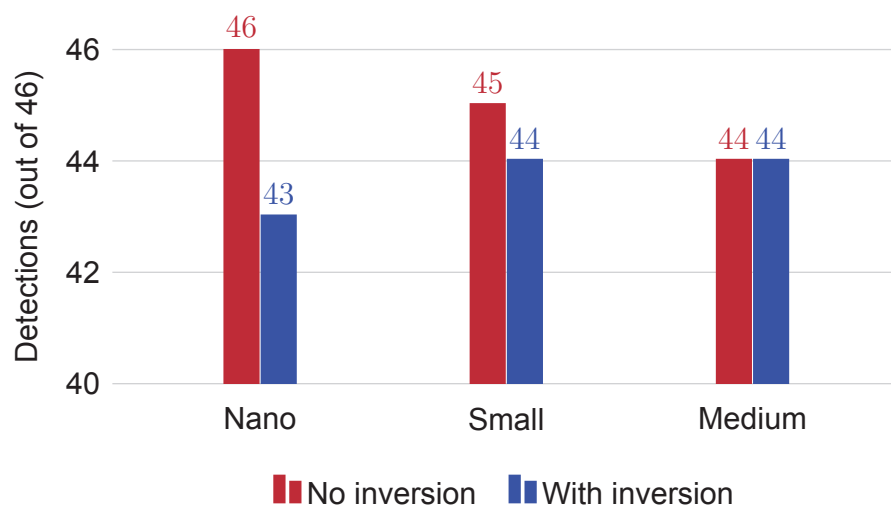


Figure B.5: False positives as a function of synthetic data volume for YOLO11m models. Augmentation combined with large synthetic datasets leads to a substantial increase in false positives.



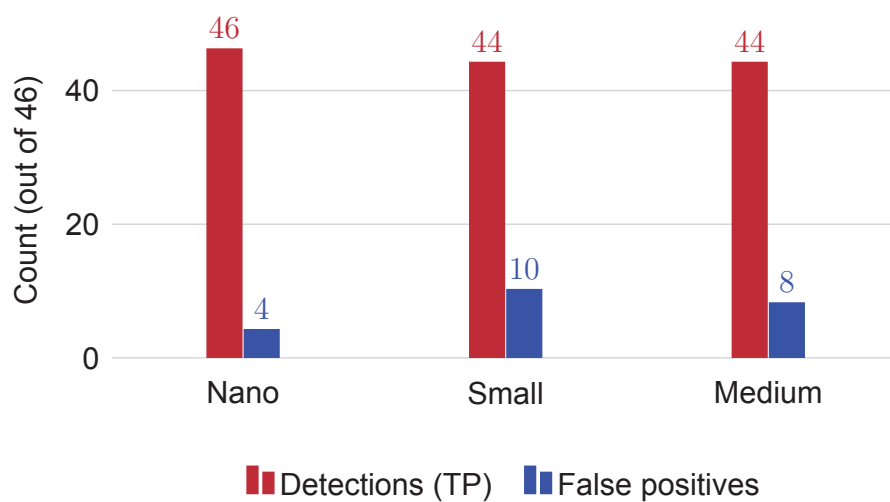
B.3 Effect of inverted training data

Figure B.6: Effect of inverted training data on YOLO11 detection performance for different model sizes. Each pair compares models trained under identical conditions, differing only in the use of inverted images.



B.4 Detection model size

Figure B.7: Comparison of YOLO11 model sizes trained on real data with augmentation. Recall is scaled by the test set size (46 images) for direct comparison.



B.5 Overall results

Table B.1: Summary of YOLO11 segmentation experiments evaluated on a fixed test set of 46 real images. True positives (TP) denote the number of correctly detected Data Matrix Codes. All synthetic samples correspond exclusively to dot-peened markings.

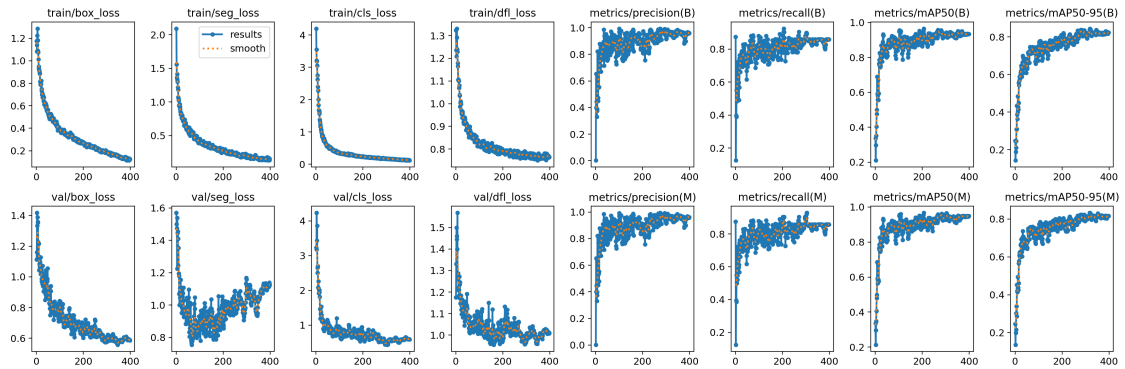
#	Model	Size	Training data	Aug.	Detections	FP
1	YOLO11	Medium	Real only	No	43	5
2	YOLO11	Medium	Real only	Yes	44	8
3	YOLO11	Medium	Real + 250 synthetic (dot-peened)	No	43	5
4	YOLO11	Medium	Real + 250 synthetic (dot-peened)	Yes	42	6
5	YOLO11	Medium	Real + 500 synthetic (dot-peened)	No	42	6
6	YOLO11	Medium	Real + 500 synthetic (dot-peened)	Yes	42	6
7	YOLO11	Medium	Real + 1000 synthetic (dot-peened)	No	43	11
8	YOLO11	Medium	Real + 1000 synthetic (dot-peened)	Yes	42	11
9	YOLO11	Medium	Real + duplicated inverted data	No	42	3
10	YOLO11	Medium	Real + inverted (polarity)	No	44	4
11	YOLO11	Medium	Real + inverted (polarity)	Yes	44	2
12	YOLO11	Small	Real only	Yes	44	10
13	YOLO11	Small	Real + 250 synthetic	No	43	5
14	YOLO11	Small	Real + 250 synthetic + inverted	No	44	2
15	YOLO11	Small	Real + 250 synthetic + inverted	Yes	43	4
16	YOLO11	Small	Real + inverted (polarity)	Yes	44	3
17	YOLO11	Nano	Real only	Yes	46	3
18	YOLO11	Nano	Real + inverted (polarity)	Yes	43	1
19	YOLO11	Nano	Real + 250 synthetic + inverted	Yes	44	2
20	YOLO11	Nano	Real + inverted (polarity)	No	42	2

B.6 Final model specifications and results

Table B.2: Training configuration and data augmentation settings used for YOLO11 segmentation models.

Category	Parameter	Value / Description
Training setup	Dataset	only_real_data.yaml
	Epochs	400
	Image size	640 × 640
	Batch size	4
	Workers	4
	Device	GPU 0
	Random seed	42
General augmentation	Augmentation	Enabled
	Brightness (HSV-V)	±15% variation
	Hue / Saturation	Disabled (grayscale imagery)
Geometric augmentation	Translation	Disabled
	Scaling	±10%
	Horizontal flip	Probability 0.25
	Vertical flip	Probability 0.10
Disabled augmentations	Mosaic	Disabled
	MixUp	Disabled
	Copy-paste	Disabled
	Random erasing	Disabled
	Shear	Disabled
	Perspective	Disabled
	Color shifts	Disabled (except brightness)
Training control	Early stopping	Patience of 80 epochs
	Training plots	Enabled
	Verbose output	Enabled
Run metadata	Run name	y11s_aug
	Output directory	models/runs/segment

Figure B.8: Training results of the final model used for this project (YOLO11 segmentation-nano).



C Geometric Rectification

C.1 Morphological mask cleaning



Figure C.1: Mask without morphological cleaning. Visible spurious artifacts, attached with pixel bridge.



(a) Dilation



(b) Erosion



(c) Morphological Closing before and after



(d) Morphological Opening before and after

Figure C.2: Morphological closing and opening process (from [13]).



Figure C.3: Mask after morphological cleaning, cleaned from spurious artifacts.

C.2 Douglas-Peucker simplification

Figure C.4: Brief explanatory visualization of the Douglas-Peucker simplification algorithm (from [23]).

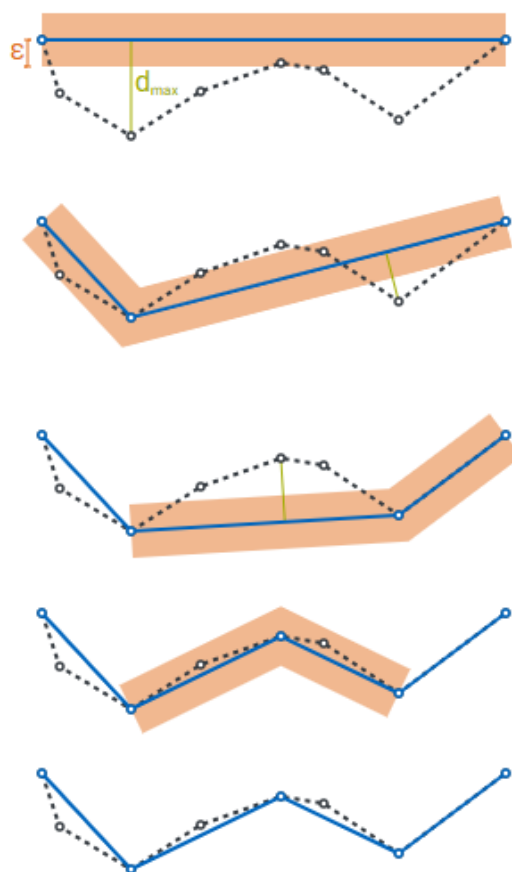
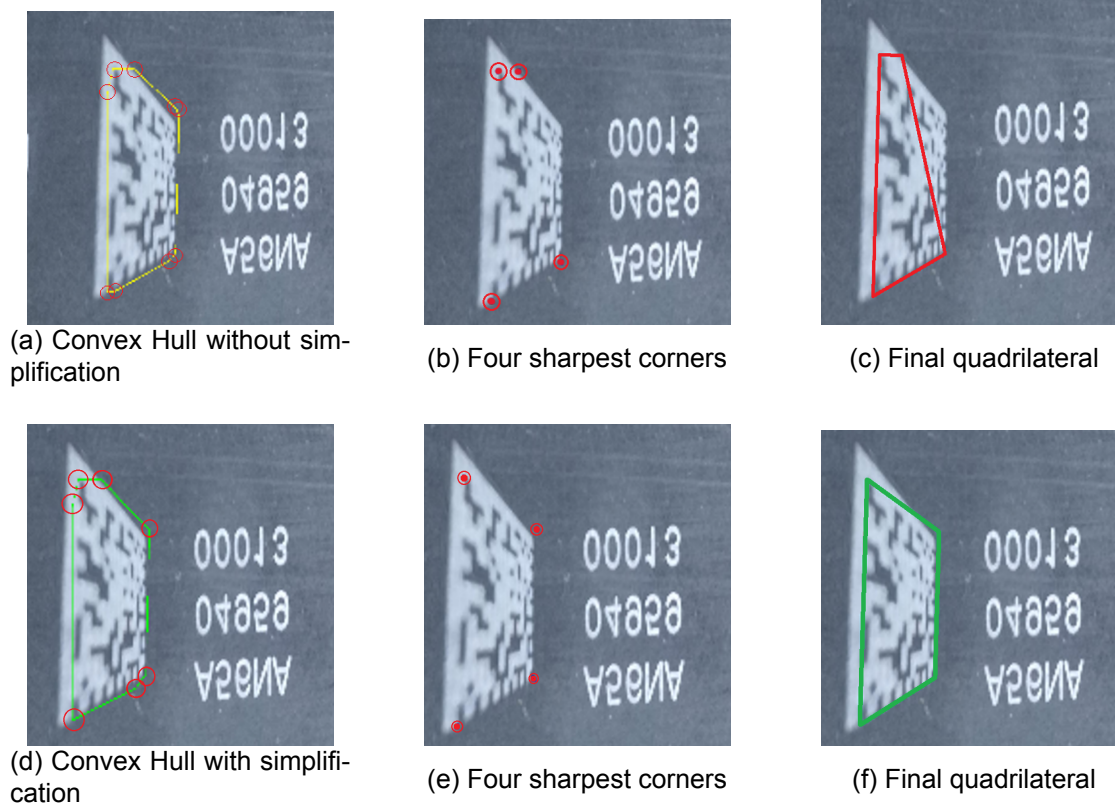
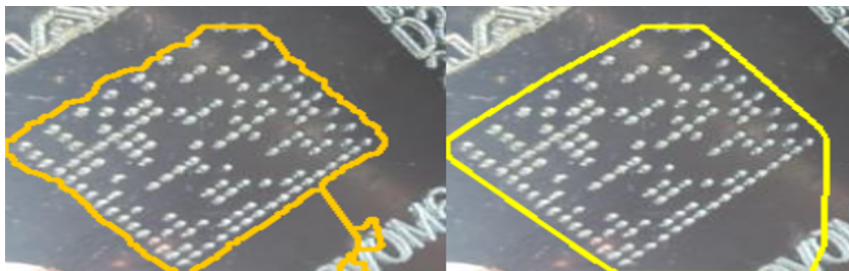


Figure C.5: Use of DP simplification to remove close corners to avoid crops with insufficient amounts of code.



C.3 Convex Hull

Figure C.6: Convex Hull result of a mask. The mask has not undergone the Morphological cleaning step to showcase the function of this step through exaggeration.



D Decoding library

Table D.1: Qualitative comparison of widely used open-source barcode decoding libraries for Data Matrix symbols. The assessment is based on published benchmarks and practical observations reported in the literature [21, 24, 25, 26].

Library	Primary target	DMC accuracy	Efficiency	Notes
libdmtx	Data Matrix	High	Slow	Specialized DMC decoder; robust on clean inputs, computationally expensive, particularly on high-resolution or low-contrast imagery.
ZXing-CPP	Multi-format	Moderate	Very fast	Optimized implementation; efficient but less reliable for challenging DMCs.
ZBar	Multi-format	Low to moderate	Moderate	General-purpose barcode reader; limited optimization for DMCs and reduced robustness on industrial markings.

E Process Visualization and Errors

Figure E.1: Progression of an image through successive stages of the pipeline.

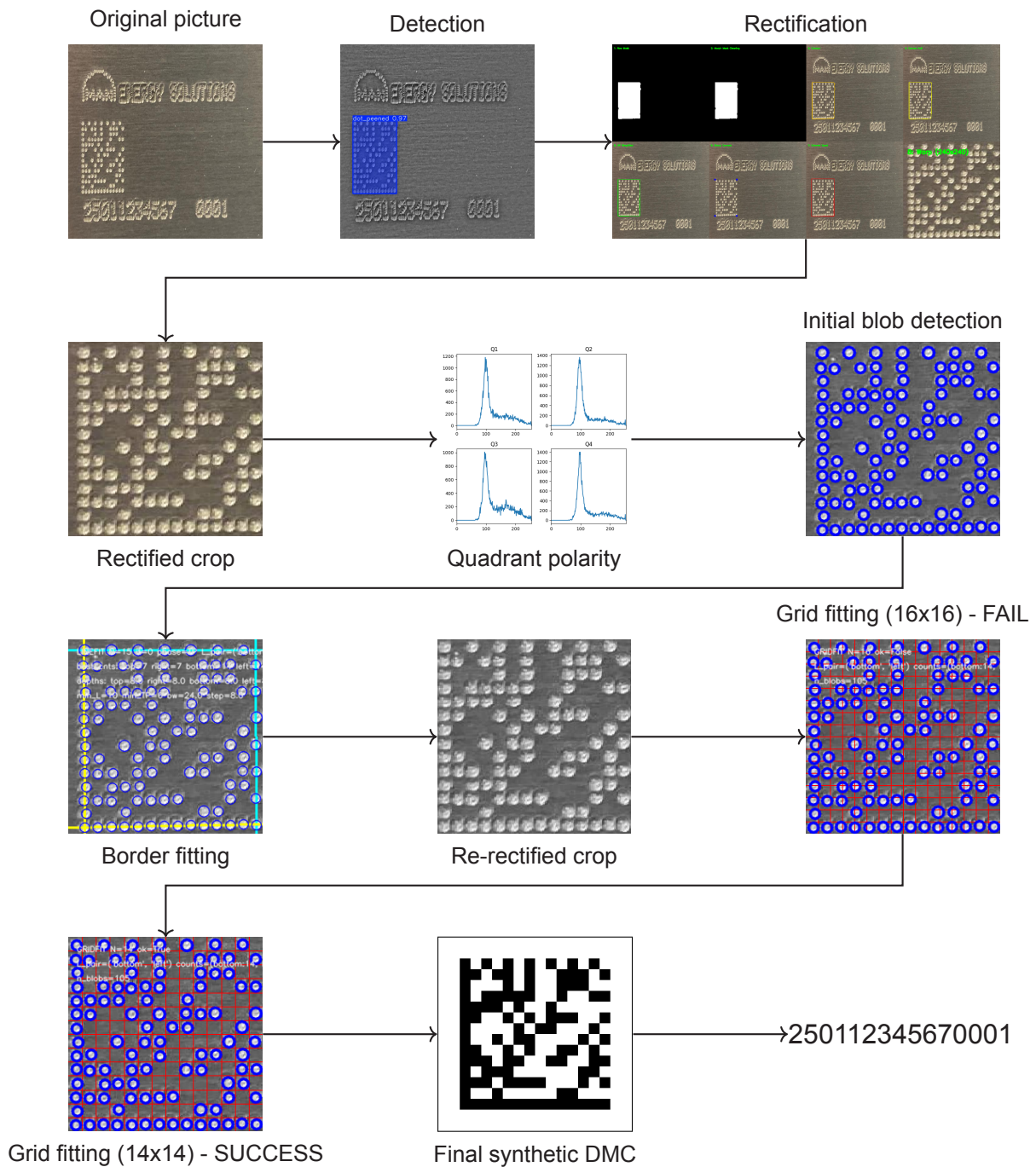


Figure E.2: Unsuccessful processing. The detection mask included non-code artifacts, that were included as blobs and made the Border fitting step to select false positive line. The Grid fitting stage rejected the crop.

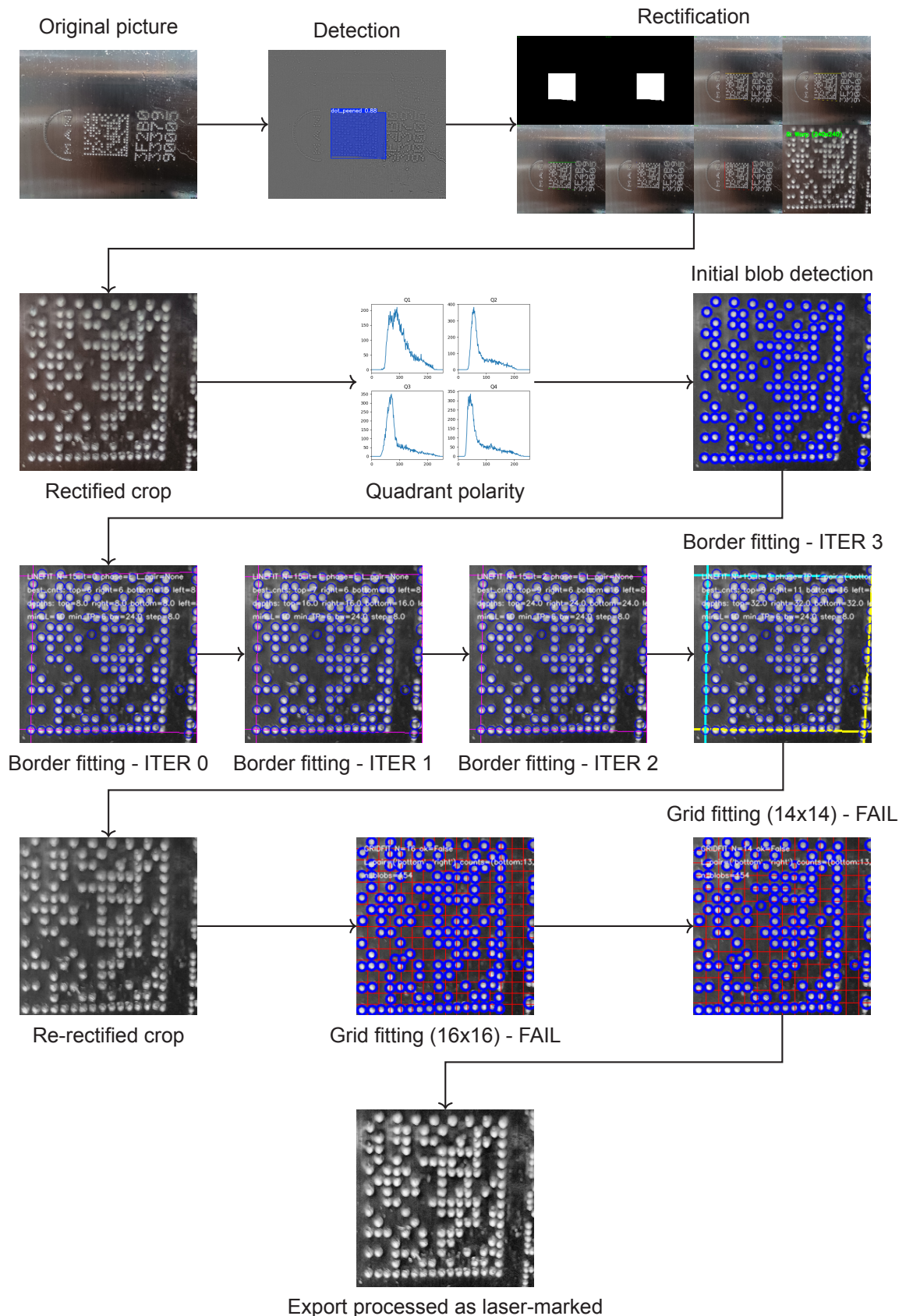


Figure E.3: Rectification failure caused by irregular segmentation mask geometry, leading to unstable quadrilateral estimation and truncation of the symbol region after perspective warping.



Figure E.4: Polarity normalization failure caused by illumination on the left side, causing its inversion and the dot artifacts to turn darker than the background, leading to unsuccessful blob detection.

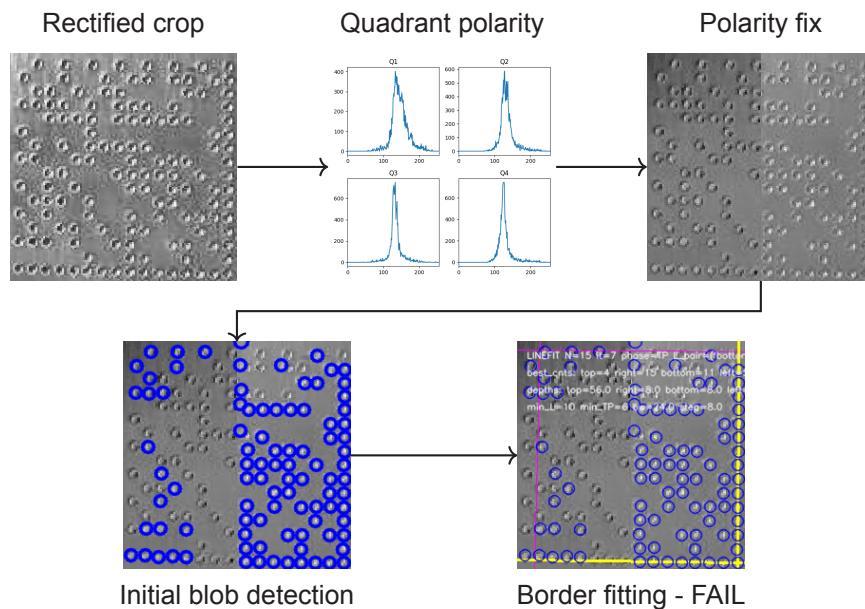


Figure E.5: Examples of rectified laser-marked DMCs that were not successfully decoded by the decoder. Although the symbols appear visually complete, decoding fails due to a combination of reduced module contrast, local intensity ambiguity, and structural degradation. These failure cases illustrate that geometric rectification alone is not sufficient for reliable decoding, consistent with the decoding failure mechanisms discussed in Section 5.1.4.

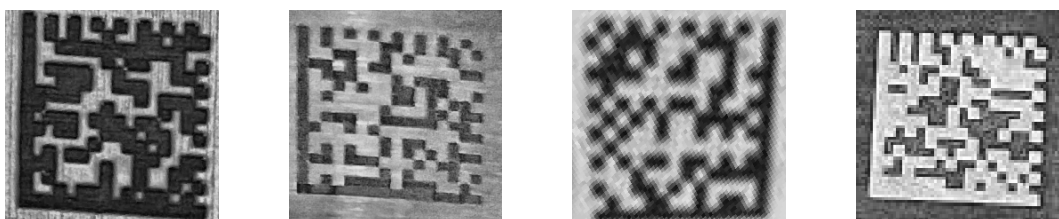
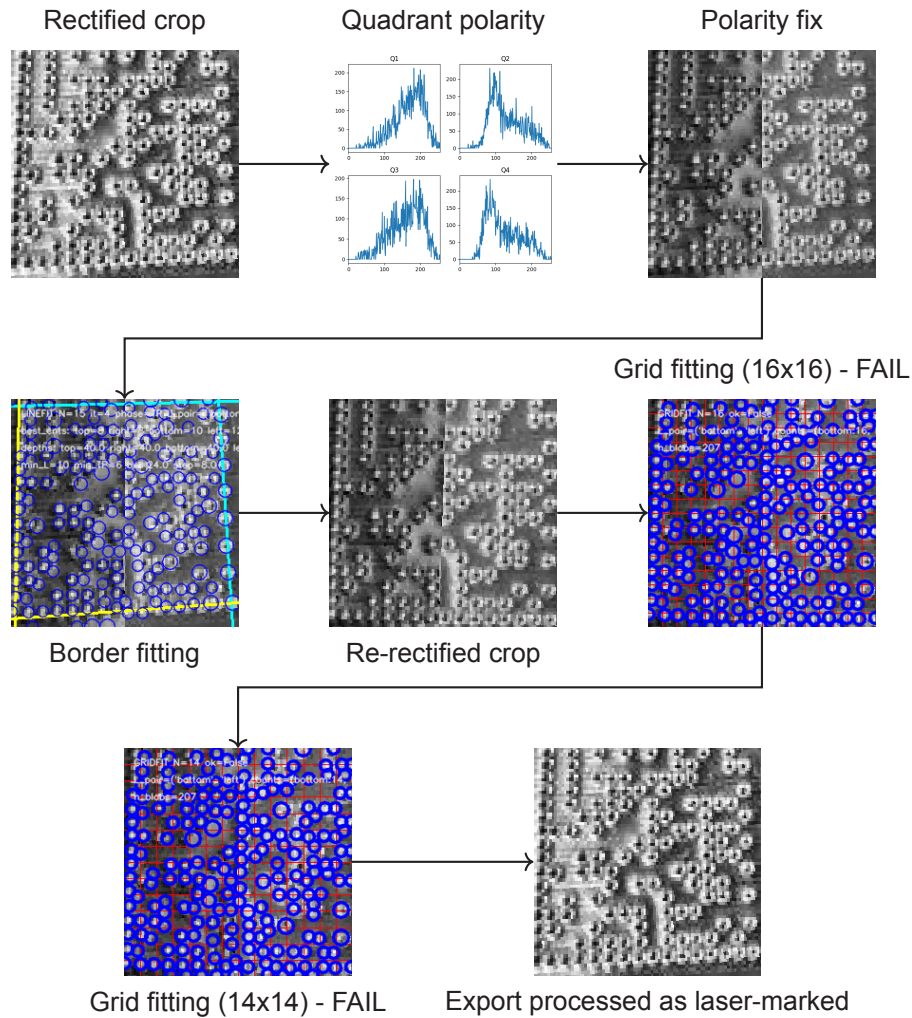


Figure E.6: Polarity normalization successfully inverting the left side of the crop, so the blobs appear lighter than the background. After re-rectification, the immense illumination across the crop is creating false positive blobs resulting to failure during the grid fitting stage. The final crop is processed and exported as a laser-marked DMC.



Technical
University of
Denmark

Building 324
2800 Kgs. Lyngby
Tlf. 45 25 30 31

www.compute.dtu.dk